

Table of Contents

[ScanDefTopic](#)

[scanabout.1.2](#)

[scanabout.1.3](#)

[scanabout.1.4](#)

[scanapps.5.1](#)

[scanapps.5.2](#)

[scanapps.5.3](#)

[scanapps.5.4](#)

[scanapps.5.5](#)

[scanapps.5.6](#)

[scanprog.3.01](#)

[scanprog.3.02](#)

[scanprog.3.03](#)

[scanprog.3.04](#)

[scanprog.3.05](#)

[scanprog.3.06](#)

[scanprog.3.07](#)

[scanprog.3.08](#)

[scanprog.3.09](#)

[scanprog.3.10](#)

[scanprog.3.11](#)

[scanprog.3.12](#)

[scanprog.3.13](#)

[scanprog.3.14](#)

[scanprog.3.15](#)

[scanprog.3.16](#)

[scanprog.3.17](#)

[scanprog.3.18](#)

[scanprog.3.19](#)

[scanprog.3.20](#)[scanprog.3.21](#)[scanprog.3.22](#)[scanprog.3.23](#)[scanprog.3.24](#)[scansyntax.4.1](#)[scansyntax.4.2](#)[scansyntax.4.3](#)[scansyntax.4.4](#)[scansyntax.4.5](#)[scantheory.2.01](#)[scantheory.2.02](#)[scantheory.2.03](#)[scantheory.2.04](#)[scantheory.2.05](#)[scantheory.2.06](#)[scantheory.2.07](#)[scantheory.2.08](#)[scantheory.2.09](#)[scantheory.2.10](#)

ScanDefTopic



About SCAN

About SCAN

Scan testing is a technique that configures a device's circuitry so that it can be tested using scan chains. These scan chains make nodes and test points more accessible for testing. The HSD1000, UltraPin800, and UltraPin1600 SCAN hardware conducts scan testing by sending serial data into the device and capturing that data efficiently.

✓ **Note:**

In this section, the upper-case "SCAN" refers to the use of specialized UltraFLEX system software and hardware to support features that cannot be supported using standard digital parallel vector memory hardware. Lower-case "scan" refers simply to standard scan test techniques and terminology.

This section includes the following topics:

- [Features](#)
- [Key Terms](#)
- [SCAN Software Licensing Requirements](#)

SCAN information includes the following sections:

- [SCAN Theory of Operation](#)
- [SCAN Programming](#)
- [SCAN Syntax Reference](#)
- [SCAN Examples](#)

Related Topics

- [Scan](#) (HSD1000 hardware specifications)
- [About the HSD1000](#)
- [HSD1000 DIB Design Guidelines](#)

- [Scan](#) (UltraPin800 hardware specifications)
- [About the UltraPin800](#)
- [UltraPin800 DIB Design Guidelines](#)
- [Scan](#) (UltraPin1600 hardware specifications)
- [About the UltraPin1600](#)
- [UltraPin1600 DIB Design Guidelines](#)

<

>

scanabout.1.2

<	>
-----------------	-----------------

About SCAN : Features

Features

The SCAN hardware has the following features:

- User configurable SCAN capability for:
 - Up to 32 scan chains per HSD1000 board
 - Up to 64 scan chains per UltraPin800 board
 - Up to 128 scan chains per UltraPin1600 board
- Up to 1.5G scan cycles with the Extended three-bit SCAN type and up to 3.0G scan cycles with the X24 two-bit SCAN type using vector memory data sharing across multiple channels

Note:

Some of the available memory for scan chains is used for hardware overhead, SRM download images, timing and levels images, and all of the VM patterns, including the one used to burst the scan chains. The exact amount of memory space used for these purposes depends on the specific application. See SCAN Vector Depth and VM Depth Licensing.

- Operation up to 500Mbps on HSD1000 and UltraPin800
- Operation up to 550Mbps on UltraPin1600 (Note that SCAN patterns are supported only in single and dual modes; quad mode is not supported.)

- [Scan patterns are supported by the full set of pattern tools: pattern compiler, editor, debugger, and reverse-compiler.](#)
- [Flexible scan pin programming. A pattern can have many scan pins. Simultaneous scan-in and scan-out on separate channels is supported.](#)
- [Editing and debugging of scan vectors in the PatternTool.](#)

For SCAN hardware specifications, see the following topics:

- [Scan](#) in the HSD1000 specifications
- [Scan](#) in the UltraPin800 specifications
- [Scan](#) in the UltraPin1600 specifications

scanabout.1.3

<

>

About SCAN : Key Terms

Key Terms

Boundary scan	A test technique that involves creating a long scan chain to test the I/O elements on the periphery of the device. (See also: JTAG)
Capture memory	Very large fail capture memory. In SCAN, it can be used to capture per pin fails and fail data per cycle. You can move fail data from the capture memory to the host DSPs for fail processing. You can also conduct this capturing and moving concurrently.
defect	A physical anomaly that occurs during the IC manufacturing process. A defect may cause the device to no longer comply with its functional or parametric design specifications.
DFT	Design-for-test. DFT adds logic or other circuitry to a device design to enhance its testability.
fault	The logical or functional result of a defect.
fault model	A description of the faulty behavior in a specific circuit characteristic or specification.
History RAM	A read only memory that captures data about the state of the pattern generator and digital channels when a pattern runs.
JTAG	Joint Test Action Group. This industry group promoted the industry standard for the IEEE Standard Test and Boundary Scan Architecture (IEEE Std. 1149.1) test interface.

internal scan	A test technique that uses existing flip-flops and latches in the design to support serial shifting of test data into the device through scan chains to test the device's internal logic circuits.
parallel vector	A parallel vector specifies data on all pins in the pattern (a "normal" pattern vector, as opposed to a scan vector). When loaded in VM, parallel vectors require three bits of VM space for every pin-cycle (as opposed to a UltraFLEX SCAN-compiled scan vector that can require as little as one bit for a pin-cycle).
path delay fault model	Fault model used to test faulty path timing behavior such as a slow-to-rise or slow-to-fall pathways. This model can be used to verify the timing specification of an internal circuit.
SCAN	The use of specialized UltraFLEX system software and hardware to support features that cannot be supported using standard digital parallel vector memory hardware.
scan cell	One element of a scan chain such as flip-flops and latches that support scan testing. Also called scan node.
scan chain	During scan testing, a collection of scan cells are connected together in a shift register configuration to form a scan chain. Test pattern data is shifted into and out of the scan chains through device scan in and scan out pins.
scan chain position	An index specifying the position of a scan cell within a scan chain. The positions are specified relative to the node at the scan out pin for the scan chain.
scan cycle	One tester cycle during a scan vector; corresponds to a single shift of the scan chain.
scan-in (pin)	A device input pin that is used to drive scan test pattern data to load a device scan chain.
scan index	The relative cycle number within a scan vector, starting with an index of 0 for the first cycle. This is generally related to scan cell positions within a scan chains, but the exact relationship depends upon a number of factors. HRAM information is generally tester-cycle related, and a single tester cycle will not correspond with a single scan chain position on all scan-in pins (unless all scan chains are exactly the same length), so HRAM displays use scan index rather than scan chain position.

scan insertion	A device design process where all sequential design elements such as flip-flops and latches are modified to support internal scan testing.
scan node	See scan cell.
scan-out (pin)	A device output pin that is used to compare scan test pattern data being unloaded from a device scan chain
scan pattern or scan test pattern	An IG-XL pattern file that uses the scan microcode to indicate tester cycles where scan test data is shifted into (and out of) a device under test.
scan pin	A DUT pin connected to one end of one or more scan chains.
scan shift	The process of simultaneously loading and unloading test pattern data to and from device scan chains. Pattern data is shifted (or "scanned") into the scan chains through scan in pins. Data is shifted (or "scanned") out of the device through scan out pins.
scan vector	A single state of all the latches in a scan chain. Also a vector in a pattern file that specifies pattern data that is applied serially to certain pins referred to as scan pins. These vectors normally supply pattern data for the scan chains within a device. Often, special tester hardware is used for executing scan vectors to minimize the amount of tester memory needed to store the data for the vector.
serial vector	A serial vector applies drive or compare data serially to a device. See also scan vector.
SRM	Subroutine Memory. A fast-access memory that allows random access during pattern bursts. SRM executes pattern loops, subroutines, and memory test and instrument-specific microcodes.
stuck-at fault model	Stuck-at fault model is used to test the structural characteristics of an integrated circuit. This model is based on shorting transistor gate connections and wire routing to power and ground. This fault model is the most commonly used for internal scan testing.
timing modes	Timing modes specify edge generation configuration for a given pattern burst. For HSD1000 and UltraPin800, IG-XL supports SCAN for a maximum data rate of 250Mbps for single timing mode and 500Mbps for dual timing mode.

transition delay fault model	This model tests the rise and fall specification of specific gates as opposed to point-to-point IC pathways. Similar to path delay fault model.
UltraFLEX databus	The databus that connects the computer running IG-XL to the tester hardware.
VM	Vector Memory. UltraFLEX VM stores SCAN vector data, parallel vector data, and on-board pin level data.

< >	
	2015 Teradyne, Inc. All rights reserved.

scanabout.1.4

<

>

About SCAN : SCAN Software Licensing Requirements

SCAN Software Licensing Requirements

IG-XL enforces a SCAN license on any digital board uses scan pins. You must have one license for each digital instrument board that uses the SCAN hardware and software.

☒ **Note:**

On UltraPin800 and UltraPin1600, a SCAN license is sold in a bundle of 128 pins, that is, the part number contains two 64 pin licenses. IG-XL enforces them as two 64 pin blocks.

Digital Board SCAN License

License Type	License Name	Enabling Points	Comments
SCAN	HSD_Scan	1 (on) or 0 (off)	This license enables the use of scan pins. One license is required per 64 pins that are used in patterns compiled in scan mode. This license is not board specific. For example, three licenses are required for a test program that has two patterns using scan mode, where those patterns use a total of 130 unique pins. When enforcing this license, IG-XL counts the number of channels that require a SCAN license on a channel by channel basis. So, IG-XL requires the same number of licenses for two SCAN channels on each of three instrument boards, as for six SCAN channels on one instrument board.

IG-XL only counts channels connected to DUT pins.

IG-XL does not require additional licenses if the test program aggregates multiple channels to increase scan chain depth.

IG-XL uses the same license key for all HSD1000, UltraPin800, and UltraPin1600 instrument boards when determining whether sufficient SCAN licenses are available in the test system.

Available SCAN vector depth depends on the licensed VM depth. See SCAN Vector Depth and VM Depth Licensing.

In an offline simulated or current configuration file, this license appears as follows:

```
<LICENSE_ENABLERS>
```

```
#Off-line enabling levels
```

```
#FEATURE          ENABLED LEVEL  LICENSE COUNT  LEGAL ENABLING POINTS
```

```
HSD_Scan          1            4            1,0
```

```
</LICENSE_ENABLERS>
```

SCAN Vector Depth and VM Depth Licensing

The number of SCAN vectors available is dependent on the instrument's VM depth license (HSD_MemoryDepth). Since SCAN is typically performed using three-bit SCAN mode and a dual 1X pattern, and since scan data is encoded to support two bits of drive and two bits of receive data per minor vector, the number of SCAN vectors available is generally the same as that described by the VM depth license enabling point.

For example, with a 64M vector license, 32M vectors is normally available for dual 1X patterns, but one two-channel SCAN chain has 64M vectors available due to the encoding. If the test program aggregates four channels for that SCAN chain, then 128M vectors are available. Other channel aggregations increase SCAN depth similarly.

2X pin mode and UltraPin1600's quad mode is not available for SCAN.

The following table lists the available SCAN vector depth based on the VM depth license.

☑ Note:

Some of the available memory for scan chains is used for hardware overhead, SRM download images, timing and levels images, and all of the VM patterns, including the one used to burst the scan chains. The exact amount of memory space used for these purposes depends on the specific application.

Available SCAN Vector Depth by VM Depth License

SCAN Vector Depth by VM Depth License (in MVectors)									
SCAN Mode	Bits	PatGen Mode	8	16	32	64	128	256	512 ¹
Parallel	6	Single	2	4	8	16	32	64	128
		Dual	4	8	16	32	64	128	256
X2	3	Single	4	8	16	32	64	128	256
		Dual	8	16	32	64	128	256	512
X4	3	Single	8	16	32	64	128	256	512
		Dual	16	32	64	128	256	512	1024
X8	3	Single	16	32	64	128	256	512	1024
		Dual	32	64	128	256	512	1024	2048
Extended	3	Single	24	48	96	192	384	768	1536
		Dual	48	96	192	384	768	1536	3072
X3	2	Single	6	12	24	48	96	192	384
		Dual	12	24	48	96	192	384	768
X6	2	Single	12	24	48	96	192	384	768
		Dual	24	48	96	192	384	768	1536
X12	2	Single	24	48	96	192	384	768	1536
		Dual	48	96	192	384	768	1536	3072
X24	2	Single	48	96	192	384	768	1536	3072
		Dual	96	192	384	768	1536	3072	6144

¹ This VM depth license is not available on HSD1000 and UltraPin800. It is only valid on UltraPin1600.

For more information, see:

- [HSD1000 Software Licensing Requirements](#)
- [UltraPin800 Software Licensing Requirements](#)

•	UltraPin1600 Software Licensing Requirements
•	IG-XL Licensing

scanapps.5.1

<	>
-----------------	-----------------

SCAN Examples

SCAN Examples

This section contains program examples for the SCAN instrument. These IG-XL workbook examples show all IG-XL data, VBT code, and patterns necessary for a complete running program.

Workbook Examples

- | |
|---|
| <ul style="list-style-type: none">SCAN X2 Single Mode Programming |
|---|

Related Topics

- | |
|---|
| <ul style="list-style-type: none">SCAN Programming |
| <ul style="list-style-type: none">TTN: Scan Test Fail Translation and Diagnosis |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scanapps.5.2

<	>
-----------------	-----------------

SCAN Examples : SCAN X2 Single Mode Programming

SCAN X2 Single Mode Programming

This workbook example shows how to program the HSD1000 to test a SN74BCT8245A scan test device on one site using an X2 single mode SCAN pattern. This example uses one HSD1000 board in slot 16.

This workbook example includes the following program components:

- | | |
|---|-----------------|
| • | DUT Connections |
| • | DataTool Sheets |
| • | Patterns |
| • | VBT Code |

<	>
-----------------	-----------------

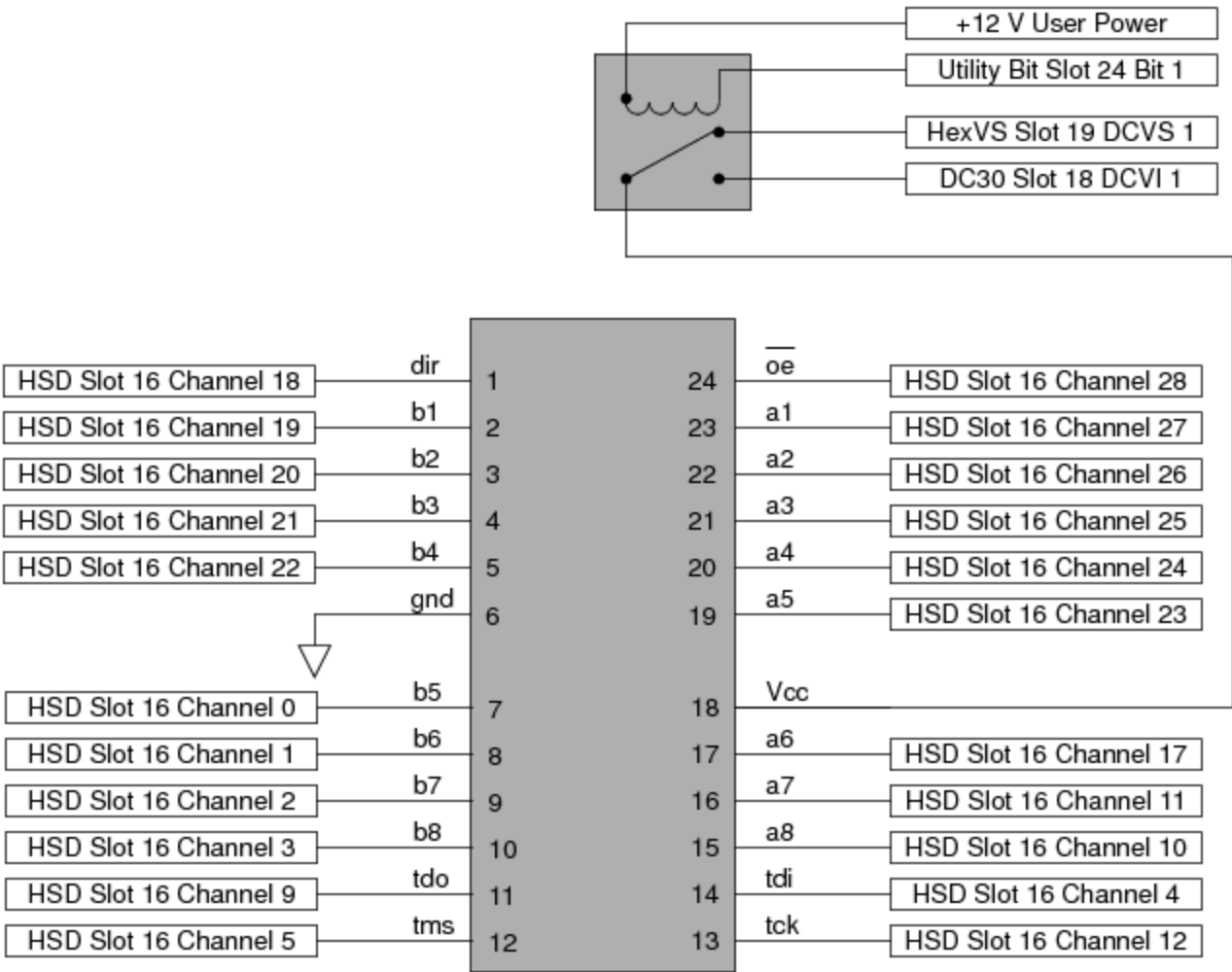
scanapps.5.3

<

>

SCAN Examples : [SCAN X2 Single Mode Programming](#) : DUT Connections

DUT Connections



HSD1000 Connections for Scan Device Site 0

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scanapps.5.4

<	>
-----------------	-----------------

SCAN Examples : SCAN X2 Single Mode Programming : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- | | |
|---|----------------------|
| • | Pin Map Sheet |
| • | Channel Map Sheet |
| • | Pin Levels Sheet |
| • | Global Specs Sheet |
| • | DC Specs Sheet |
| • | AC Specs Sheet |
| • | Time Sets Sheet |
| • | Edge Sets Sheet |
| • | Test Instances Sheet |
| • | Flow Table Sheet |

Pin Map Sheet

Pin Map

Group Name	Pin Name	Type	Comment
	dir	Input	direction control
	b1	I/O	port B bit 1
	b2	I/O	port B bit 2
	b3	I/O	port B bit 3
	b4	I/O	port B bit 4
	gnd	Gnd	ground
	b5	I/O	port B bit 5
	b6	I/O	port B bit 6
	b7	I/O	port B bit 7
	b8	I/O	port B bit 8
	tdo	Output	scan output - test data output
	tms	Input	test mode select
	tck	Input	test clock
	tdi	Input	scan input - test data input
	a8	I/O	port A bit 8
	a7	I/O	port A bit 7
	a6	I/O	port A bit 6
	Vcc	Power	+5 Volt supply
	a5	I/O	port A bit 5
	a4	I/O	port A bit 4
	a3	I/O	port A bit 3
	a2	I/O	port A bit 2
	a1	I/O	port A bit 1
	oe	Input	output enable
	DUT_Power	Utility	
portA	a8	I/O	port A bits 8 - 1
portB	b8	I/O	port B bits 8 -1
scan_in	tdi	Input	
scan_out	tdo	Output	
ctrls	dir	Input	
ctrls	oe		
scan_ctrls	tms	Input	
scan_ctrls	tck		
non_scan	tms	I/O	
non_scan	tck		

Pin Map

Group Name	Pin Name	Type	Comment	
non_scan	dir			
non_scan	oe			
non_scan	portA			
non_scan	portB			
allpins	tms	I/O		
allpins	tck			
allpins	tdi			
allpins	tdo			
allpins	dir			
allpins	oe			
allpins	portA			
allpins	portB			
odds	dir	I/O		
odds	b2			
odds	b4			
odds	b5			
odds	b7			
odds	tdo			
odds	tck			
odds	a8			
odds	a6			
odds	a5			
odds	a3			
odds	a1			
evens	b1	I/O		
evens	b3			
evens	b6			
evens	b8			
evens	tms			
evens	tdi			
evens	a7			
evens	a4			
evens	a2			
evens	oe			
ports	portA	I/O		
ports	portB			

Channel Map Sheet

Channel Map

DIB ID:

View Mode: Signal

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
dir	1	I/O	16.ch18	HSD Chan
b1	2	I/O	16.ch19	HSD Chan
b2	3	I/O	16.ch20	HSD Chan
b3	4	I/O	16.ch21	HSD Chan
b4	5	I/O	16.ch22	HSD Chan
gnd	6	Gnd		Ground
b5	7	I/O	16.ch0	HSD Chan
b6	8	I/O	16.ch1	HSD Chan
b7	9	I/O	16.ch2	HSD Chan
b8	10	I/O	16.ch3	HSD Chan
tdo	11	I/O	16.ch9	HSD Chan
tms	12	I/O	16.ch5	HSD Chan
tck	13	I/O	16.ch12	HSD Chan
tdi	14	I/O	16.ch4	HSD Chan
a8	15	I/O	16.ch10	HSD Chan
a7	16	I/O	16.ch11	HSD Chan
a6	17	I/O	16.ch17	HSD Chan
Vcc	18	DCVS	19.sense1	DCVS Source 1
a5	19	I/O	16.ch23	HSD Chan
a4	20	I/O	16.ch24	HSD Chan
a3	21	I/O	16.ch25	HSD Chan
a2	22	I/O	16.ch26	HSD Chan
a1	23	I/O	16.ch27	HSD Chan
oe	24	I/O	16.ch28	HSD Chan
DUT_Power		Utility	24.util64	

[Pin Levels Sheet](#)

Defined using formulas:

Pin Levels

Pin/Group	Seq.	Parameter	Value
ports		Vil	=_Vil
ports		Vih	=_Vih
ports		Vol	=_VolA
ports		Voh	=_VohA
ports		Vt	=_Vt
ports		Vcl	-0.5
ports		Vch	3.5
ports		VoutLoTyp	=_VolA
ports		VoutHiTyp	=_VohA
ports		DriverMode	Largeswing-VT
scan_in		Vil	=_Vil
scan_in		Vih	=_Vih
scan_in		Vt	=_Vt
scan_in		Vcl	-0.5
scan_in		Vch	3.5
scan_in		DriverMode	Largeswing-HiZ

Pin Levels

Pin/Group	Seq.	Parameter	Value
scan_out		V _{ol}	= _VolA
scan_out		V _{oh}	= _VohA
scan_out		V _t	= _Vt
scan_out		V _{cl}	-0.5
scan_out		V _{ch}	3.5
scan_out		V _{outLoTyp}	= _VolA
scan_out		V _{outHiTyp}	= _VohA
scan_out		DriverMode	Largeswing-VT
ctrls		V _{il}	= _Vil
ctrls		V _{ih}	= _Vih
ctrls		V _t	= _Vt
ctrls		V _{cl}	-0.5
ctrls		V _{ch}	3.5
ctrls		DriverMode	Largeswing-HiZ
scan_ctrls		V _{il}	= _Vil
scan_ctrls		V _{ih}	= _Vih
scan_ctrls		V _t	= _Vt
scan_ctrls		V _{cl}	-0.5
scan_ctrls		V _{ch}	3.5
scan_ctrls		DriverMode	Largeswing-HiZ
Vcc		V _{ps}	= _Vcc_val
Vcc		I _{sc}	=400*mA
Vcc		T _{delay}	=20*ms

Formulas evaluated:

Pin Levels

Pin/Group	Seq.	Parameter	Value
ports		Vil	0.8
ports		Vih	3
ports		Vol	0.5
ports		Voh	2.4
ports		Vt	1.4
ports		Vcl	-0.5
ports		Vch	3.5
ports		VoutLoTyp	0.5
ports		VoutHiTyp	2.4
ports		DriverMode	Largeswing-VT
scan_in		Vil	0.8
scan_in		Vih	3
scan_in		Vt	1.4
scan_in		Vcl	-0.5
scan_in		Vch	3.5
scan_in		DriverMode	Largeswing-HiZ

Pin Levels

Pin/Group	Seq.	Parameter	Value
scan_out		Vol	0.5
scan_out		Voh	2.4
scan_out		Vt	1.4
scan_out		Vcl	-0.5
scan_out		Vch	3.5
scan_out		VoutLoTyp	0.5
scan_out		VoutHiTyp	2.4
scan_out		DriverMode	Largeswing-VT
ctrls		Vil	0.8
ctrls		Vih	3
ctrls		Vt	1.4
ctrls		Vcl	-0.5
ctrls		Vch	3.5
ctrls		DriverMode	Largeswing-HiZ
scan_ctrls		Vil	0.8
scan_ctrls		Vih	3
scan_ctrls		Vt	1.4
scan_ctrls		Vcl	-0.5
scan_ctrls		Vch	3.5
scan_ctrls		DriverMode	Largeswing-HiZ
Vcc		Vps	5
Vcc		Isc	0.4
Vcc		Tdelay	0.02

Global Specs Sheet

Global Specs

Symbol	Job	Value	Comment
Vcl_default	-	1.E+00	Detector clamp voltage low
Vch_default		5.E+00	Detector clamp voltage high
Vph_default		5.E+00	Hi-V pin voltage high
Iph_default		0	Hi-V pin current limit
Tpr_default		16.E-06	Hi-V pin rise time
Tdelay		1.E-03	

DC Specs Sheet

DC Specs



Symbol	Value	Selector		Commercial			Zero_specs
		Name	Val	Typ	Min	Max	Typ
Vih	3	Func_nom	Typ	3	2	#N/A	0
Vihh	10	Func_nom	Typ	10	#N/A	#N/A	0
Vil	0.8	Func_nom	Typ	0.8	#N/A	#N/A	0
VohA	2.4	Func_nom	Typ	2.4	2.4	#N/A	0
VohB	2.4	Func_nom	Typ	2.4	2.4	#N/A	0
VolA	0.5	Func_nom	Typ	0.5	#N/A	0.5	0
VolB	0.5	Func_nom	Typ	0.5	#N/A	0.5	0
Vt	1.4	Func_nom	Typ	1.4	0.75	1.25	0
Icc_hi	3.6E-09	Func_nom	Typ	3.6E-09	#N/A	7.5E-09	0
Vcc_val	5	Func_nom	Typ	5	4.5	5.5	0
Icc_dis	0.0015	Func_nom	Typ	0.0015	#N/A	3.5E-09	0
Icc_lo	0.035	Func_nom	Typ	0.035	#N/A	0.052	#N/A

AC Specs Sheet

AC Specs



Symbol	Value	Selector		Normal	
		Name	Val	Typ	Comment
tSU	0.00000002	Normal	Typ	0.00000002	
tSU_TMS	0.00000002	Normal	Typ	0.00000002	
tH	0.00000002	Normal	Typ	0.00000002	
tH_TMS	0.00000002	Normal	Typ	0.00000002	
PERIOD	0.00000001	Normal	Typ	0.00000001	
tW	0.00000002	Normal	Typ	0.00000002	
tPLH	0.000000013	Normal	Typ	0.000000013	

Time Sets Sheet

Time Sets

Timing Mode: Master Timeset Name: Time Domain:

Time Set	Cycle	Pin/Group		Setup	Edge Set	Comment
	Period	Name	Clock Period			
tset_scan	100.E-09	portA		i/o	es_scan	
tset_scan	100.E-09	portB		i/o	es_scan	
tset_scan	100.E-09	tck		i/o	es_scan	
tset_scan	100.E-09	dir		i/o	es_scan	
tset_scan	100.E-09	oe		i/o	es_scan	
tset_scan	100.E-09	tms		i/o	es_scan	
tset_scan	100.E-09	tdi		i/o	es_scan	
tset_scan	100.E-09	tdo		i/o	es_scan	

Edge Sets Sheet

Edge Sets

Timing Mode: Time Domain:

		Data		Drive				Compare			Edge Resolution
Pin/Group	Edge Set	Src	Fmt	On	Data	Return	Off	Mode	Open	Close	Mode
portA	es_scan	PAT	NR	0	0		0	Edge	75.E-09	95.E-09	Machine
portB	es_scan	PAT	NR	0	0		0	Edge	75.E-09	95.E-09	Machine
tck	es_scan	PAT	RL	0	20.E-09	70.E-09	0	Off			Machine
dir	es_scan	PAT	NR	0	0		0	Off			Machine
oe	es_scan	PAT	NR	0	0		0	Off			Machine
tms	es_scan	PAT	NR	0	5.E-09		0	Off			Machine
tdi	es_scan	PAT	NR	0	0		0	Off			Machine
tdo	es_scan	PAT	NR	0	0		10.E-09	Edge	75.E-09	97.E-09	Machine

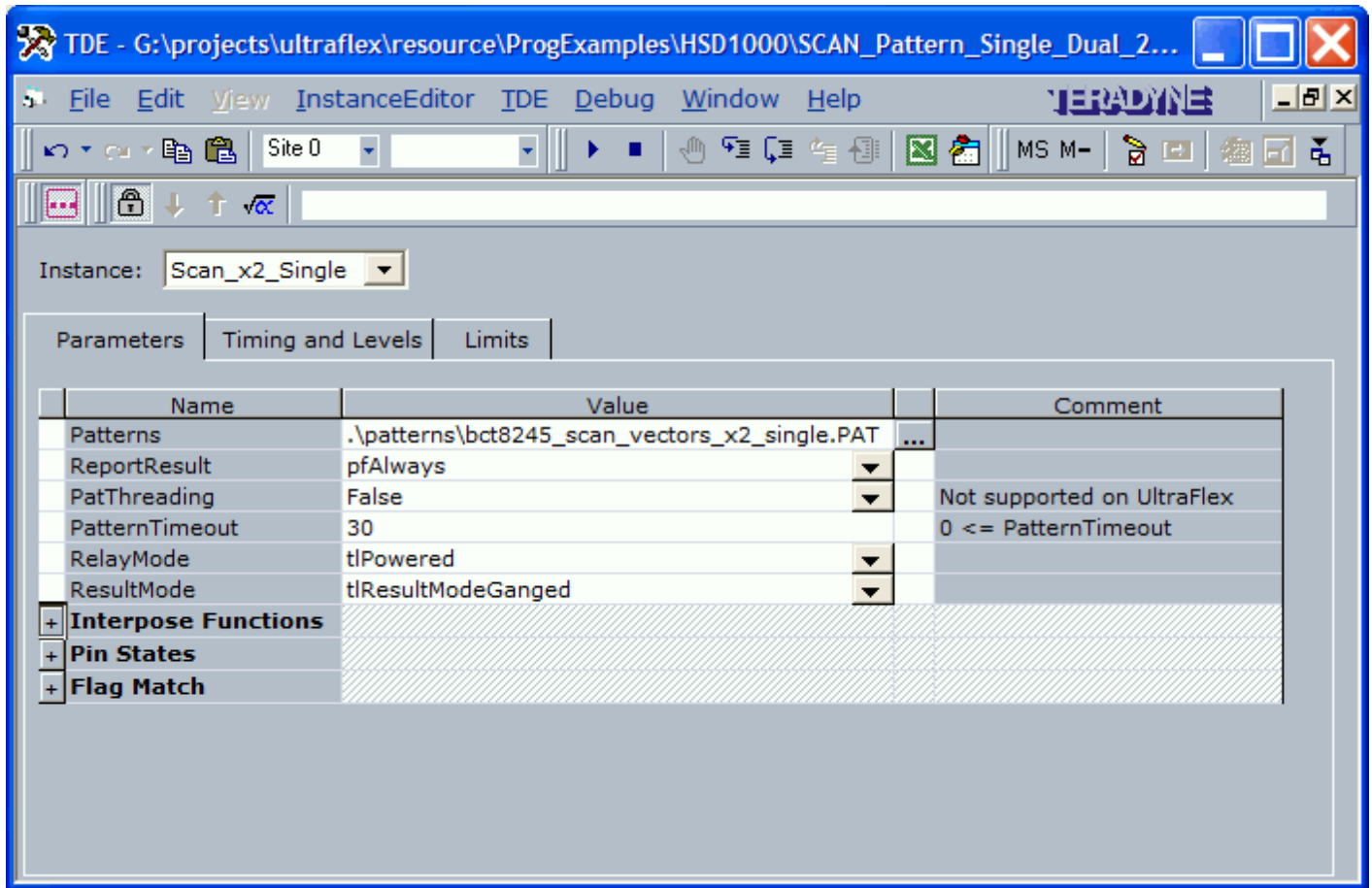
Test Instances Sheet

Test Instances



Test Name	Test Procedure			DC Specs		AC Specs		Sheet Parameters			
	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels	Mixed Signal Timing Overlay
Scan_x2_Single	VBT	Functional_T		Commercial	Func_nom	Normal	Normal	Tsets_Single	Esets_Single	NomLevel	

The Scan_x2_Single test instance has the following test-specific parameters and values as shown in the Instance Editor:



Flow Table Sheet

Flow Table



		Gate			Command				Limits		Bin Number		Sort Number		
Label	Enable	Job	Part	Env	Opcode	Parameter	TName	TNum	LoLim	HiLim	Pass	Fail	Pass	Fail	Result
					Test	Scan_x2_Single		200				2		20	Fail
					stop										

2015 Teradyne, Inc. All rights reserved.

scanapps.5.5

	
---	---

SCAN Examples : SCAN X2 Single Mode Programming : Patterns

Patterns

bct8245_scan_vectors_x2_single.atp Pattern File

// .atp Scan File Example using the TI8245 Device

```
digital_inst = hsdm;
opcode_mode = single;
import tset tset_scan;
scan_type = x2;
// tdi = scan input pin
// tdo = scan output pin
//-----

scan_pins = {
    tdi,
    tdo
}

vm_vector ( $tset, dir:S, oe:S, portA:S, portB:S, tdi:S, tdo:S, tck:S, tms:S )
{
    begin_scan_vec:
    //Initialize TAP-controller state to Shift-IR
        > tset_scan X X .X .X 1 X 1 1; //1. Extra cycle to get TCK started
        > tset_scan X X .X .X 1 X 1 1; //2. Start Reset Sequence
        > tset_scan X X .X .X 1 X 1 1; //3. Reset Sequence
    repeat 30 > tset_scan X X .X .X 1 X 1 1; //4. Reset Sequence
```

```

> tset_scan X X .X .X 1 X 1 1; //5. Reset Sequence
> tset_scan X X .X .X 1 X 1 1; //6. Test-Logic-Reset
> tset_scan X X .X .X 1 X 1 0; //7. To Run-Test/Idle
> tset_scan X X .X .X 1 X 1 1; //8. To Select-DR-Scan
> tset_scan X X .X .X 1 X 1 1; //9. To Select-IR-Scan
> tset_scan X X .X .X 1 X 1 0; //10. To Capture-IR
> tset_scan X X .X .X 1 X 1 0; //11. To Shift-IR

```

```
// Shift in transport test sequence (1001101100)
```

```
// and then bypass instructions (11111111)
```

```

> tset_scan X X .X .X 1 H 1 0 ; //12. Shift in test seq 1001101100
> tset_scan X X .X .X 0 L 1 0 ; //13. thru inst reg (8-bit long)
> tset_scan X X .X .X 0 X 1 0 ; //14.
> tset_scan X X .X .X 1 X 1 0 ; //15.
> tset_scan X X .X .X 1 X 1 0 ; //16.
> tset_scan X X .X .X 0 X 1 0 ; //17.
> tset_scan X X .X .X 1 X 1 0 ; //18.
> tset_scan X X .X .X 1 X 1 0 ; //19.
> tset_scan X X .X .X 0 H 1 0 ; //20. Expect test seq 1001101100 to
> tset_scan X X .X .X 0 L 1 0 ; //21. come out at TDO 8 vectors later
> tset_scan X X .X .X 1 L 1 0 ; //22. Shift in BYPASS inst 11111111
> tset_scan X X .X .X 1 H 1 0 ; //23.
> tset_scan X X .X .X 1 H 1 0 ; //24.
> tset_scan X X .X .X 1 L 1 0 ; //25.
> tset_scan X X .X .X 1 H 1 0 ; //26.
> tset_scan X X .X .X 1 H 1 0 ; //27.
> tset_scan X X .X .X 1 L 1 0 ; //28.
> tset_scan X X .X .X 1 L 1 1 ; //29. Last TDO bit - To Exit1-IR
> tset_scan X X .X .X 1 X 1 1 ; //30. To Update-IR -> bypass mode
> tset_scan X X .X .X 1 X 1 0 ; //31. To Run-Test/Idle

```

```
//-----
```

```
// Test the BYPASS register, length = 1
```

```

> tset_scan X X .X .X 1 X 1 1 ; //32. To Select-DR-Scan
> tset_scan X X .X .X 1 X 1 0 ; //33. To Capture-DR
> tset_scan X X .X .X 1 X 1 0 ; //34. To Shift-DR -> capture bypass data 0

```

```
// Shift in transport test sequence (1001101100) for DUT in BYPASS test
```

```
// Shift out bypass data 0, shift out test seq HLLHHLHLL thru BYPASS reg
```

```
scan_setup
```

```
> tset_scan X X .X .X 0 L 1 0 ; //35-44.
```

```
scan 10
```

```
(si tdi:S > 1001101100)
```

```
(so tdo:S > LHLHHLHHL);
```

```
> tset_scan X X .X .X 1 L 1 1 ; //45. Last TDO bit - To Exit1-DR
```

```
> tset_scan X X .X .X 1 X 1 1 ; //46. To Update-DR -> noop
```

```
> tset_scan X X .X .X 1 X 1 0 ; //47. To Run-Test/Idle
```

```
> tset_scan X X .X .X 1 X 1 1 ; //48. To Select-DR-Scan
```

```
> tset_scan X X .X .X 1 X 1 1 ; //49. To Select-IR-Scan
```

```
> tset_scan X X .X .X 1 X 1 0 ; //50. To Capture-IR
```

```
// Shift in instruction sample (00000010)
```

```
> tset_scan X X .X .X 1 X 1 0 ; //51. To Shift-IR
```

```
> tset_scan X X .X .X 0 X 1 0 ; //52. IR0
```

```
> tset_scan X X .X .X 1 X 1 0 ; //53. IR1
```

```
> tset_scan X X .X .X 0 X 1 0 ; //54. IR2
```

```
> tset_scan X X .X .X 0 X 1 0 ; //55. IR3
```

```
> tset_scan X X .X .X 0 X 1 0 ; //56. IR4//SMS added
```

```
> tset_scan X X .X .X 0 X 1 0 ; //57. IR5//SMS added
```

```
> tset_scan X X .X .X 0 X 1 0 ; //58. IR6
```

```
> tset_scan X X .X .X 0 X 1 1 ; //59. IR7 To Exit1-IR.Last IR bit
```

```
> tset_scan X X .X .X 1 X 1 1 ; //60. To Update-IR -> sample mode
```

```
// sample/boundary reg/length = 18
```

```
> tset_scan X X .X .X 1 X 1 0 ; //61. To Run-Test/Idle
```

```
> tset_scan X X .X .X 1 X 1 1 ; //62. To Select-DR-Scan
```

```
> tset_scan X X .X .X 1 X 1 0 ; //63. To Capture-DR
```

```
> tset_scan X X .X .X 1 X 1 0 ; //64. To Shift-DR
```

```
// Shift in test seq 1001101100
```

```
// Shift out test seq 1001101100 thru BSR (18-bit long)
```

```
scan_setup
```

```
> tset_scan X X .X .X 0 L 1 0 ; //65-92.EXTENDED Scan from 27-->28
```

```
scan 26
```

```
(si tdi:S > 10011011001111111111111111111111)
```



```
(so tdo:S > XXXXXXXXXXXXXXXXXXXXHLLHHLHH);
```

```
> tset_scan X X .X .X 1 L 1 0 ; //SMS last scan cycle needs go to parallel to get even on scan
> tset_scan X X .X .X 1 L 1 1 ; //93 Last TDO bit - To Exit1-DR
> tset_scan X X .X .X 1 X 1 1 ; //94. To Update-DR
> tset_scan X X .X .X 1 X 1 0 ; //95. To Run-Test/Idle
> tset_scan X X .X .X 1 X 1 1 ; //96. To Select-DR-Scan
> tset_scan X X .X .X 1 X 1 1 ; //97. To Select-IR-Scan
> tset_scan X X .X .X 1 X 1 0 ; //98. To Capture-IR
```

```
// Shift in instruction sample (00000010)
```

```
> tset_scan X X .X .X 1 X 1 0 ; //99. To Shift-IR
> tset_scan X X .X .X 0 X 1 0 ; //100. IR0
> tset_scan X X .X .X 1 X 1 0 ; //101. IR1
> tset_scan X X .X .X 0 X 1 0 ; //
repeat 3 > tset_scan X X .X .X 0X 1 0 ; //102-105. IR2-IR5
> tset_scan X X .X .X 0 X 1 0 ; //106. IR6
> tset_scan X X .X .X 0 X 1 1 ; //107. IR7
> tset_scan X X .X .X 1 X 1 1 ; //108. To Update-IR -> sample mode
```

```
// sample/boundary reg/length = 18
```

```
> tset_scan X X .X .X 1 X 1 0 ; //109 To Run-Test/Idle
> tset_scan X X .X .X 1 X 1 1 ; //S1.1 To Select-DR-Scan
> tset_scan X X .X .X 1 X 1 0 ; //S1.2 To Capture-DR
> tset_scan X X .X .X 1 X 1 0 ; //S1.3 To Shift-DR
```

```
//SMS added a NOP vector to get even
```

```
> tset_scan X X .X .X 1 X 0 0 ; //NOP
```

```
//-----
```

```
// Shift in ser stimulus #110101010111111101 thru BSR (18-bit long)
```

```
scan_setup
```

```
> tset_scan X X .X .X 0 L 1 0 ; //S1.4-21 EXTENDED Scan from 17-->18
```

```
scan 16
```

```
(si tdi:S > 1010101011111111)
```

```
(so tdo:S > XXXXXXXXXXXXXXXX); //O8-O1 = 10101010 and OE = 0
```

```
> tset_scan X X .X .X 0 X 1 0 ; //SMS last scan cycle needs go to
//parallel to get even on scan
> tset_scan X X .X .X 1 X 1 1 ; //S1.22 Last TDO bit.
//To Exit1-DR, DIR = 1
```

```

> tset_scan X X .X .X 1 X 1 1 ; //S1.23 To Update-DR
> tset_scan X X .X .X 1 X 1 0 ; //S1.24 To Run-Test/Idle
// Instructions for DUT in EXTEST
> tset_scan X X .X .X 1 X 1 1 ; //S1.25 To Select-DR-Scan
> tset_scan X X .X .X 1 X 1 1 ; //S1.26 To Select-IR-Scan
> tset_scan X X .X .X 1 X 1 0 ; //S1.27 To Capture-IR
// Shift in instruction sample (00000000)
> tset_scan X X .X .X 1 X 1 0 ; //S1.28 To Shift-IR
> tset_scan X X .X .X 0 X 1 0 ; //S1.29 IR0
repeat 6 > tset_scan X X .X .X 0 X 1 0 ; //S1.32-S1.35 IR3-IR6
> tset_scan X X .X .X 0 X 1 1 ; //S1.36 IR7 To Exit-IR
> tset_scan X X .X .X 1 X 1 1 ; //S1.37 To Update-IR -> EXTEST mode
// extest/boundary reg/length = 18
> tset_scan X X .X .X 1 X 1 0 ; //S1.39 To Run-Test/Idle
> tset_scan X X .X .X 1 X 1 1 ; //S2R1.1 To Select-DR-Scan
> tset_scan X X .X .X 1 X 1 0 ; //S2R1.2 To Capture-DR

// capture parallel inputs and parallel outputs (ser stim #1)
//(DIR = 1 from ser stim -> porta=in, portb=out)
> tset_scan 1 0 10101010 HLHLHLHL 1 X 1 0 ; //S2R1.3 To Shift-DR

// Shift in ser stim #2 and shift out parallel stim #1 from S2R1.3
scan_setup
> tset_scan X X .X .X 0 L 1 0 ; // S2R1.4-21EXTENDED Scan from 17-->18
scan 16
(si tdi:S > 0101010111111111) //shift in new O8-O1 = 01010101 and OE = 0
(so tdo:S > XXXXXXXXHLHLHLHL) ; //shift out old I8-I0 = 10101010
//followed by OE = 0, DIR = 1

> tset_scan X X .X .X 0 L 1 0 ; //SMS last scan cycle needs go to
//parallel to get even on scan
> tset_scan X X .X .X 1 H 1 1 ; //S2R1.22 Last TDO bit.
//To Exit1-DR, DIR = 1
> tset_scan X X .X .X 1 X 1 1 ; //S2R1.23 To Update-DR
> tset_scan X X .X .X 1 X 1 0 ; //S2R1.24 To Run-Test/Idle
// > tset_scan X X .X .X 1 X 1 0 ; //S2R1.24 To Run-Test/Idle(padded)
> tset_scan X X .X .X 1 X 1 1 ; //R2.1 To Select-DR-Scan
> tset_scan X X .X .X 1 X 1 0 ; //R2.2 To Capture-DR

```

```

// capture parallel inputs and parallel outputs (ser stim #2)
// for A8-A1 (DIR = 0 from ser stim)
> tset_scan 0 1 01010101 LHLHLHLH 1 X 1 0 ; //R2.3 To Shift-DR

//SMS added a NOP vector to get even
> tset_scan X X .X .X 0 X 0 0 ; //NOP
scan_setup
> tset_scan X X .X .X 0 L 1 0 ; //R2.4-21 EXTENDED Scan from 17-->18
scan 16
// Shift out parallel stim #2 from R2.3
(si tdi:S > 0101010111111111) //shift in new O8-O1 = 10101010 and OE = 0
(so tdo:S > XXXXXXXXLHLHLHLH) ; //shift out old I8-I0 = 01010101
//followed by OE = 1, DIR = 1

> tset_scan X X .X .X 0 H 1 0 ; //SMS last scan cycle needs go to
//parallel to get even on scan
> tset_scan X X .X .X 1 L 1 1 ; //R2.22 Last TDO bit. To Exit1-DR, DIR = 1
> tset_scan X X .X .X 1 X 1 1 ; //R2.23 To Update-DR
> tset_scan X X .X .X 1 X 1 0 ; //R2.24 To Run-Test/Idle
> tset_scan X X .X .X 1 X 1 1 ; //all-done, reinitialize TAP to reset

repeat 5 > tset_scan X X .X .X 1 X 1 1 ; //all-done,
//reinitialize TAP to reset

end_scan_vec:
> tset_scan X X .X .X 1 X 1 1 ; //all-done, reinitialize TAP to reset
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A

```

```
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
> tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
halt > tset_scan X X .X .X X X X X ; //stop... end of: scan_mod_A
}
```

< >		
	2015 Teradyne, Inc. All rights reserved.	

scanapps.5.6

<	>
-----------------	-----------------

SCAN Examples : [SCAN X2 Single Mode Programming](#) : VBT Code

VBT Code

This [SCAN](#) program example does not require VBT code.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scanprog.3.01

<	>
----------------------	----------------------

SCAN Programming

SCAN Programming

This section describes programming for SCAN.

The following topics are available:

- | | |
|---|---|
| • | SCAN Testing Procedure Overview |
| • | Selecting SCAN Type |
| • | Allocating SCAN Channels |
| • | Creating Scan Patterns |
| • | Compiling Scan Patterns |
| • | Debugging Using PatternTool |
| • | Using Deep Scan Capture |

For more information, see [SCAN Examples](#).

<	>
----------------------	----------------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scanprog.3.02

	
---	---

[SCAN Programming](#) : SCAN Testing Procedure Overview

SCAN Testing Procedure Overview

Scan testing is performed using digital patterns. You create a pattern that contains the pin data that sets the device scan chains to the desired states. The most common scan test patterns (internal scan) are created using ATPG tools. Although the following procedure describes a manual process for creating a scan test application, all steps in the procedure except the manual generation of test patterns apply to ATPG-based scan test applications.

To develop SCAN testing programs, follow this procedure:

1. [Choose the required SCAN type.](#)

[See Selecting SCAN Type.](#) You must do this step before you design the DIB.

2. [Select the digital channels that are to be connected to the DUT.](#)

[See Allocating SCAN Channels.](#)

3. [Set up your scan test using a functional template.](#)

[See the HSD1000, UltraPin800, or UltraPin1600 instrument section on creating a functional test.](#)

4. [Create ASCII text pattern file using scan pattern syntax.](#)

[See Creating Scan Patterns.](#) Be sure to specify the SCAN type determined in step 1.

5. [Compile the ASCII pattern file to create the binary pattern file.](#)

[See Compiling Scan Patterns](#). Do not use the `-scan_type` parallel compiler option if you want to create SCAN-compatible binary pattern files. SCAN optimization will have no effect if the scan patterns are not SCAN-compatible.

6. [Debug the program using the PatternTool and HRAM](#).

[See Debugging Using PatternTool](#).



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scanprog.3.03

	
---	---

SCAN Programming : Selecting SCAN Type

Selecting SCAN Type

To support SCAN-compatible test patterns, carefully consider scan type. If scan patterns are compiled and loaded as parallel vectors, then selecting your scan type is not necessary. However, maximum scan pattern depth in parallel mode is restricted to 128Mcycles on HSD1000 and UltraPin800, and 256Mcycles on UltraPin1600.

Note:

During channel assignment you must consider the scan type; otherwise, the scan patterns will not load properly.

Before You Begin

Before you select the SCAN type for your application, you must first know your system information and device requirements:

- The number of HSD1000, UltraPin800, or UltraPin1600 digital channel boards in the test system

See the system configuration file, CurrentConfig.txt, in the /tester directory for this information.

- The number of device scan chains for your application

- The required scan test pattern memory depth

- The type of scan chains (two-bit or three-bit)

If you need a don't care (X) state, use three-bit scan chains.

Three-Bit Scan Chain Applications

This procedure below uses the following reference table to help you select the SCAN type for your application based on the HSD1000, UltraPin800, or UltraPin1600 board’s scan chain configurations. If you use UltraPin800 boards, you have twice as many available channels (128), so the maximum number of scan chains is doubled. If you use UltraPin1600 boards, you have four times as many available channels (256), so the maximum number of scan chains is quadrupled.

Note that the maximum scan data rate for HSD1000 and UltraPin800 is 250Mbps in single PatGen mode and 500Mbps in dual PatGen mode. For UltraPin1600, the maximum scan data rate is 275Mbps in single mode and 550Mbps in dual mode.

 Note:

This section focuses on using SCAN types that support three-bit scan chains because a majority of scan applications require three-bit scan chains. The HSD1000, and UltraPin800, and UltraPin1600 SCAN supports two-bit scan chains. The SCAN type selection procedure described below, however, focuses only on the three-bit scan chain application.

Three-bit Scan Chain Configurations per Digital Board

SCAN Type	Maximum Number of Scan Chains (N _{max_chains})			Maximum Scan Depth (MVectors) (D _{max_depth}) ¹	
	HSD1000	UltraPin800	UltraPin1600	HSD1000/ UltraPin800	UltraPin1600
				(Single, Dual)	(Single, Dual)
X2	32	64	128	128, 256	256, 512
X4	16	32	64	256, 512	512, 1024
X8	8	16	32	512, 1024	1024, 2048
Extended	5	10	20	768, 1536	1536, 3072

¹ Some of the available memory for scan chains is used for hardware overhead, SRM download images, timing and levels images, and all of the VM patterns, including the one used to burst the scan chains. The exact amount of memory space used for these purposes depends on the specific application.

Two-Bit Scan Chain Applications

HSD1000, UltraPin800, and UltraPin1600 SCAN also supports two-bit scan chains. The procedure below can also be used with two-bit scan chains by using the table below in place of table above. Although two-bit scan chains support deeper scan chain depth, they do not support more than two scan-out pin compare states. For more information on scan-out pin states, see Scan Data.

Note that the maximum scan data rate for HSD1000 and UltraPin800 is 250Mbps in single PatGen mode and 500Mbps in dual PatGen mode. For UltraPin1600, the maximum scan data rate is 275Mbps in single mode and 550Mbps in dual mode.

Two-bit Scan Chain Configurations per Digital Board

SCAN Type	Maximum Number of Scan Chains (N _{max_chains})			Maximum Scan Depth (MVectors) (D _{max_depth}) ¹	
				HSD1000/ UltraPin800	UltraPin1600
	HSD1000	UltraPin800	UltraPin1600	(Single, Dual)	(Single, Dual)
X3	32	64	128	192, 384	384, 768
X6	16	32	64	384, 768	768, 1536
X12	8	16	32	768, 1536	1536, 3072
X24	4	8	16	1536, 3072	3072, 6144

¹ Some of the available memory for scan chains is used for hardware overhead, SRM download images, timing and levels images, and all of the VM patterns, including the one used to burst the scan chains. The exact amount of memory space used for these purposes depends on the specific application.

Selecting SCAN Type Procedure

To determine the SCAN type for your three-bit scan chain application, follow this procedure:

1. Calculate the required number of scan chains per HSD1000, UltraPin800, or UltraPin1600 board using the following equations.

If the number of digital channel boards divides evenly into the number of scan chains for the DUT, then:

$$N_{req_chains_per_board} = \left(\frac{N_{chains_per_device}}{N_{hsd_boards}} \right)$$

(3-1)

otherwise:

$$N_{req_chains_per_board} = \frac{N_{chains_per_device}}{N_{hsd_boards}} + 1$$

(3-2)

where $N_{chains_per_device}/N_{hsd_boards}$ is rounded down to the next whole number.

2. Choose a SCAN type that supports the required number of device scan chains for each digital board given your system configuration.

To maximize the available scan chain depth, see the table above to select a SCAN type where the maximum number of scan chains is greater than or equal to the required number of device scan chains per board ($N_{\text{max_chains}} \geq N_{\text{req_chains_per_board}}$), while providing the greatest scan chain depth ($D_{\text{max_depth}}$).

For example, if $N_{\text{req_chains_per_board}} = 10$ on HSD1000, choose the X4 SCAN type. The table above shows that both X2 and X4 SCAN types support the required number of scan chains, while X8 and Extended SCAN cannot. However, X4 SCAN provides a greater scan chain depth than X2 SCAN (512Mcycles versus 256Mcycles).

3. Determine if the selected SCAN type supports the required scan chain depth.

If you know the required scan chain depth for your application ($D_{\text{required_depth}}$), use the table above to make this determination. The maximum scan chain depth must be greater than or equal to the required scan chain depth ($D_{\text{max_depth}} \geq D_{\text{required_depth}}$).

If greater scan chain depth is required, you must consider using a SCAN type that supports deeper scan chains. However, since you already selected a SCAN type to maximize scan chain depth given the required number of scan chains, your only other alternative for increasing the scan chain depth is to add digital instrument boards to your system configuration.

To determine how many more boards your system configuration requires, repeat step 2 using a greater value for $N_{\text{hsd_boards}}$ until you are able to use a SCAN type that supports your required scan chain depth.

For example, if $N_{\text{req_chains_per_board}} = 10$ on HSD1000 and $D_{\text{required_depth}} = 600\text{Mcycles}$, X4 SCAN does not support your application. To solve this problem, you must add HSD1000 boards to your configuration so that $N_{\text{req_chains_per_board}} = 8$. You can then use the X8 SCAN type, which supports eight scan chains that can be up to 1024Mcycles deep.

HSD1000 SCAN Type Selection Example

This example is based on a UltraFLEX system configuration that has four HSD1000 boards and a device that requires 40 scan chains and has 600Mcycles worth of scan test patterns. So:

- $N_{\text{hsd_boards}} = 4$
- $N_{\text{chains_per_device}} = 40$

- $D_{\text{required_depth}} = 600\text{Mcycles}$

1. Calculate the required number of scan chains per HSD1000 board.

$$\begin{aligned} N_{\text{req_chains_per_board}} &= N_{\text{chains_per_device}} / N_{\text{hsd_boards}} \\ &= 40 / 4 \\ &= 10 \end{aligned}$$

2. Choose a SCAN type that supports the required number of device scan chains for each HSD1000 board given your system configuration.

Given the four HSD1000 system configuration, the X4 SCAN type supports 40 scan chains.

3. Determine if the selected SCAN type supports the required scan chain depth.

The maximum scan chain depth supported by X4 SCAN is 512Mcycles. However,

$D_{\text{required_depth}} = 600\text{Mcycles}$, which is greater than 512Mcycles. Therefore, you cannot use X4 SCAN for this application.

The table above shows that you must use either X8 or Extended SCAN to support 600Mcycle scan chains. Since using X8 SCAN requires fewer additional HSD1000 boards, this SCAN type should be your target.

For X8 SCAN you need, at most, $N_{\text{req_chains_per_board}} = 8$.

To do this, you can change around the equation 3–1 from step 1 to determine $N_{\text{hsd_boards}}$:

$$\begin{aligned} N_{\text{hsd_boards}} &= (N_{\text{chains_per_device}} / N_{\text{req_chains_per_board}}) \\ &= (40 / 8) \\ &= 5 \end{aligned}$$

So, if you add one more HSD1000 board for a total of five HSD1000 boards in the system configuration, you can use X8 SCAN to support both the required number of scan chains (40) and the required scan chain depth (600Mcycles).

What if you needed more than 1024Mcycles of scan chain depth?

For Extended SCAN, you need, at most, $N_{\text{req_chains_per_board}} = 5$. Therefore:

$$\begin{aligned} N_{\text{hsd_boards}} &= (N_{\text{chains_per_device}} / N_{\text{req_chains_per_board}}) \\ &= (40 / 5) \\ &= 8 \end{aligned}$$

If this 40 scan chain application requires more than 1024Mcycles of scan chain depth, then you need a total of eight HSD1000 boards, and you must compile the scan patterns using the Extended SCAN type.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

scanprog.3.04

<	>
-----------------	-----------------

SCAN Programming : Allocating SCAN Channels

Allocating SCAN Channels

After determining the SCAN type for your application, follow these channel assignment rules to allocate your digital channels.

☒ **Note:**

The rules shown below are for the HSD1000 and the first eight scan regions (channels 0-63) of the UltraPin800 and UltraPin1600. The same rules apply to the UltraPin800 board's next eight scan regions (channels 64-127) by adding 64 to the channel numbers in these tables. For the UltraPin1600, add 64 to the tables' channel numbers for the next eight scan regions (channels 64-127), 128 for the next eight scan regions (channels 128-191), and 192 for the final eight scan regions (channels 192-255).

Rules for three-bit SCAN types:

- | | |
|---|---|
| • | Even-Odd SCAN Channel Rule |
| • | SCAN Channel Assignment Rules for X2 SCAN |
| • | SCAN Channel Assignment Rules for X4 SCAN |
| • | SCAN Channel Assignment Rules for X8 SCAN |
| • | SCAN Channel Assignment Rules for Extended SCAN |

Rules for two-bit SCAN types:

- [SCAN Channel Assignment Rules for X3 SCAN](#)
- [SCAN Channel Assignment Rules for X6 SCAN](#)
- [SCAN Channel Assignment Rules for X12 SCAN](#)
- [SCAN Channel Assignment Rules for X24 SCAN](#)

Once you are familiar with the SCAN channel assignment rules for your application’s selected SCAN type, you can use the following SCAN channel allocation tables to help design your DIB. During DIB design, use the channel allocation tables as a guide to which specific channels can be assigned to scan-in and scan-out pins to support your selected SCAN type. The scan-in and scan-out pins for a given scan chain do not need to be assigned to same SCAN region or adjacent pair.

 Note:

Use this scan channel assignment when designing your DIB. If you fail to follow these channel allocation rules, your SCAN-compatible patterns will not load.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

scanprog.3.05

<	>
-----------------	-----------------

SCAN Programming : Allocating SCAN Channels : Even-Odd SCAN Channel Rule

Even-Odd SCAN Channel Rule

For three-bit scan chains, scan-in pins must be assigned to even-numbered channels, and scan-out pins must be assigned to odd-numbered channels.

This rule does not apply to two-bit scan chains. With two-bit scan chains, scan-in and scan-out pins can be assigned to either even or odd channels.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scanprog.3.06

<

>

SCAN Programming : Allocating SCAN Channels : SCAN Channel Assignment Rules for X2 SCAN

SCAN Channel Assignment Rules for X2 SCAN

- Scan-in pins can be assigned to any even channels.
- Scan-out pins can be assigned to any odd channels.
- Up to four scan-in pins can be assigned to each SCAN region.
- Up to four scan-out pins can be assigned to each SCAN region.

Valid X2 SCAN Channels

SCAN Region	Available for Scan Pins							
	Scan In				Scan Out			
1	Ch0	Ch2	Ch4	Ch6	Ch1	Ch3	Ch5	Ch7
2	Ch8	Ch10	Ch12	Ch14	Ch9	Ch11	Ch13	Ch15
3	Ch16	Ch18	Ch20	Ch22	Ch17	Ch19	Ch21	Ch23
4	Ch24	Ch26	Ch28	Ch30	Ch25	Ch27	Ch29	Ch31
5	Ch32	Ch34	Ch36	Ch38	Ch33	Ch35	Ch37	Ch39
6	Ch40	Ch42	Ch44	Ch46	Ch41	Ch43	Ch45	Ch47
7	Ch48	Ch50	Ch52	Ch54	Ch49	Ch51	Ch53	Ch55

8	Ch56	Ch58	Ch60	Ch62	Ch57	Ch59	Ch61	Ch63
---	------	------	------	------	------	------	------	------

☒ **Note:**

Non-SCAN channels can be used for parallel (scan_setup) data pins.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scanprog.3.07

<

>

SCAN Programming : Allocating SCAN Channels : SCAN Channel Assignment Rules for X4 SCAN

SCAN Channel Assignment Rules for X4 SCAN

- Scan-in pins can assigned to the first and fifth channels in each SCAN region.
- Scan-out pins can be assigned to the second and sixth channels in each SCAN region.
- Up to two scan-in pins can be assigned to each SCAN region.
- Up to two scan-out pins can be assigned to each SCAN region.

Valid X4 SCAN Channels

SCAN Region	Available for Scan Pins				For Setup Pins ¹			
	Scan In		Scan Out		Scan In		Scan Out	
1	Ch0	Ch4	Ch1	Ch5	Ch2	Ch6	Ch3	Ch7
2	Ch8	Ch12	Ch9	Ch13	Ch10	Ch14	Ch11	Ch15
3	Ch16	Ch20	Ch17	Ch21	Ch18	Ch22	Ch19	Ch23
4	Ch24	Ch28	Ch25	Ch29	Ch26	Ch30	Ch27	Ch31
5	Ch32	Ch36	Ch33	Ch37	Ch34	Ch38	Ch35	Ch39
6	Ch40	Ch44	Ch41	Ch45	Ch42	Ch46	Ch43	Ch47
7	Ch48	Ch52	Ch49	Ch53	Ch50	Ch54	Ch51	Ch55

8	Ch56	Ch60	Ch57	Ch61	Ch58	Ch62	Ch59	Ch63
---	------	------	------	------	------	------	------	------

1 Setup pins are parallel pins whose pin data does not change during a scan vector.

 Note:

Non-SCAN channels can be used for parallel (scan_setup) data pins.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

scanprog.3.08

<

>

SCAN Programming : Allocating SCAN Channels : SCAN Channel Assignment Rules for X8 SCAN

SCAN Channel Assignment Rules for X8 SCAN

- Scan-in pins can be assigned to the first channel of a SCAN region.
- Scan-out pins can be assigned to the second channel of a SCAN region.

Valid X8 SCAN Channels

SCAN Region	Available for Scan Pins		For Setup Pins ¹					
	Scan In	Scan Out	Scan In			Scan Out		
1	Ch0	Ch1	Ch2	Ch4	Ch6	Ch3	Ch5	Ch7
2	Ch8	Ch9	Ch10	Ch12	Ch14	Ch11	Ch13	Ch15
3	Ch16	Ch17	Ch18	Ch20	Ch22	Ch19	Ch21	Ch23
4	Ch24	Ch25	Ch26	Ch28	Ch30	Ch27	Ch29	Ch31
5	Ch32	Ch33	Ch34	Ch36	Ch38	Ch35	Ch37	Ch39
6	Ch40	Ch41	Ch42	Ch44	Ch46	Ch43	Ch45	Ch47
7	Ch48	Ch49	Ch50	Ch52	Ch54	Ch51	Ch53	Ch55
8	Ch56	Ch57	Ch58	Ch60	Ch62	Ch59	Ch61	Ch63

¹ Setup pins are parallel pins whose pin data does not change during a scan vector.

☒ **Note:**

Non-SCAN channels can be used for parallel (scan_setup) data pins.

< >		
	2015 Teradyne, Inc. All rights reserved.	

scanprog.3.09

<

>

SCAN Programming : Allocating SCAN Channels : SCAN Channel Assignment Rules for Extended SCAN

SCAN Channel Assignment Rules for Extended SCAN

- Scan-in pins and scan-out pins must not be assigned to the same SCAN region.
- Scan-in pins can be assigned to the first and second channels of a SCAN region.
- Scan-out pins must be assigned to the second channel of a SCAN region.

☒ **Note:**

The following table shows one of many possible Extended SCAN configurations. The availability of scan channels depends on the type of scan pins assigned in a SCAN region. An extended SCAN configuration can have only one scan-out pin per SCAN region or up to two scan-in pins per SCAN region.

Valid Extended SCAN Channels

SCAN Region	Available for Scan Pins			For Setup Pins ¹								
	In	In	Out	In	In	In	In	In	In	Out	Out	Out
1	Ch0	Ch1		Ch2	Ch3	Ch4	Ch5	Ch6	Ch7			
2	Ch8	Ch9		Ch10	Ch11	Ch12	Ch13	Ch14	Ch15			
3	Ch16			Ch18	Ch20	Ch22						

4	Ch25	Ch27	Ch29	Ch31
5	Ch33	Ch35	Ch37	Ch39
6	Ch41	Ch43	Ch45	Ch47
7	Ch49	Ch51	Ch53	Ch55
8	Ch57	Ch59	Ch61	Ch63

¹ Setup pins are parallel pins whose pin data does not change during a scan vector.

☒ **Note:**

Non-SCAN channels can be used for parallel (scan_setup) data pins. Scan-out pins can be assigned to the second channel of a SCAN region.

<

>

2015 Teradyne, Inc. All rights reserved.

scanprog.3.10

<

>

SCAN Programming : [Allocating SCAN Channels](#) : SCAN Channel Assignment Rules for X3 SCAN

SCAN Channel Assignment Rules for X3 SCAN

- [Scan-in pins can be assigned to any channel.](#)
- [Scan-out pins can be assigned to any channel.](#)
- [Up to eight scan pins can be assigned to each SCAN region.](#)

Valid X3 SCAN Channels

SCAN Region	Available for Scan Pins							
1	Ch0	Ch1	Ch2	Ch3	Ch4	Ch5	Ch6	Ch7
2	Ch8	Ch9	Ch10	Ch11	Ch12	Ch13	Ch14	Ch15
3	Ch16	Ch17	Ch18	Ch19	Ch20	Ch21	Ch22	Ch23
4	Ch24	Ch25	Ch26	Ch27	Ch28	Ch29	Ch30	Ch31
5	Ch32	Ch33	Ch34	Ch35	Ch36	Ch37	Ch38	Ch39
6	Ch40	Ch41	Ch42	Ch43	Ch44	Ch45	Ch46	Ch47
7	Ch48	Ch49	Ch50	Ch51	Ch52	Ch53	Ch54	Ch55
8	Ch56	Ch57	Ch58	Ch59	Ch60	Ch61	Ch62	Ch63

 **Note:**

Non-SCAN channels can be used for parallel (scan_setup) data pins.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

scanprog.3.11

<

>

SCAN Programming : Allocating SCAN Channels : SCAN Channel Assignment Rules for X6 SCAN

SCAN Channel Assignment Rules for X6 SCAN

- Scan-in or scan-out pins can be assigned to the first, second, fourth, or fifth channels of each SCAN region.
- Up to four scan pins can be assigned to each SCAN region.

Valid X6 SCAN Channels

SCAN Region	Available for Scan Pins				For Setup Pins ¹			
1	Ch0	Ch1	Ch4	Ch5	Ch2	Ch3	Ch6	Ch7
2	Ch8	Ch9	Ch12	Ch13	Ch10	Ch11	Ch14	Ch15
3	Ch16	Ch17	Ch20	Ch21	Ch18	Ch19	Ch22	Ch23
4	Ch24	Ch25	Ch28	Ch29	Ch26	Ch27	Ch30	Ch31
5	Ch32	Ch33	Ch36	Ch37	Ch34	Ch35	Ch38	Ch39
6	Ch40	Ch41	Ch44	Ch45	Ch42	Ch43	Ch46	Ch47
7	Ch48	Ch49	Ch52	Ch53	Ch50	Ch51	Ch54	Ch55
8	Ch56	Ch57	Ch60	Ch61	Ch58	Ch59	Ch62	Ch63

¹ Setup pins are parallel pins whose pin data does not change during a scan vector.

 Note:

Non-SCAN channels can be used for parallel (scan_setup) data pins.

< >		
	2015 Teradyne, Inc. All rights reserved.	

scanprog.3.12

<

>

SCAN Programming : Allocating SCAN Channels : SCAN Channel Assignment Rules for X12 SCAN

SCAN Channel Assignment Rules for X12 SCAN

- Scan-in or scan-out pins can be assigned to the first or second channels of each SCAN region.
- Up to two scan pins can be assigned to each SCAN region.

Valid X12 SCAN Channels

SCAN Region	Available for Scan Pins		For Setup Pins ¹					
1	Ch0	Ch1	Ch2	Ch3	Ch4	Ch5	Ch6	Ch7
2	Ch8	Ch9	Ch10	Ch11	Ch12	Ch13	Ch14	Ch15
3	Ch16	Ch17	Ch18	Ch19	Ch20	Ch21	Ch22	Ch23
4	Ch24	Ch25	Ch26	Ch27	Ch28	Ch29	Ch30	Ch31
5	Ch32	Ch33	Ch34	Ch35	Ch36	Ch37	Ch38	Ch39
6	Ch40	Ch41	Ch42	Ch43	Ch44	Ch45	Ch46	Ch47
7	Ch48	Ch49	Ch50	Ch51	Ch52	Ch53	Ch54	Ch55
8	Ch56	Ch57	Ch58	Ch59	Ch60	Ch61	Ch62	Ch63

¹ Setup pins are parallel pins whose pin data does not change during a scan vector.

 **Note:**

Non-SCAN channels can be used for parallel (scan_setup) data pins.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

scanprog.3.13

<

>

SCAN Programming : Allocating SCAN Channels : SCAN Channel Assignment Rules for X24 SCAN

SCAN Channel Assignment Rules for X24 SCAN

- Scan-in or scan-out pins must be assigned to the first channels of each SCAN region.
- Only one scan-in pin or one scan-out pin can be assigned to each SCAN region.

☒ **Note:**

The following table shows one of many possible X24 SCAN configurations. The availability of scan channels depends on the type of scan pins assigned in a SCAN region.

Valid X24 SCAN Channels

SCAN Region	Available for Scan Pins	For Setup Pins ¹						
1	Ch0	Ch1	Ch2	Ch3	Ch4	Ch5	Ch6	Ch7
2	Ch8	Ch9	Ch10	Ch11	Ch12	Ch13	Ch14	Ch15
3	Ch16	Ch17	Ch18	Ch19	Ch20	Ch21	Ch22	Ch23
4	Ch24	Ch25	Ch26	Ch27	Ch28	Ch29	Ch30	Ch31
5	Ch32	Ch33	Ch34	Ch35	Ch36	Ch37	Ch38	Ch39
6	Ch40	Ch41	Ch42	Ch43	Ch44	Ch45	Ch46	Ch47
7	Ch48	Ch49	Ch50	Ch51	Ch52	Ch53	Ch54	Ch55
8	Ch56	Ch57	Ch58	Ch59	Ch60	Ch61	Ch62	Ch63

¹ Setup pins are parallel pins whose pin data does not change during a scan vector.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

scanprog.3.14

<	>
-----------------	-----------------

SCAN Programming : Creating Scan Patterns

Creating Scan Patterns

In most cases commercial off-the-shelf automated test pattern generation (ATPG) tools generate scan test patterns for complex digital devices. Normally, these ATPG scan test patterns are in a non-IG-XL format, such as STIL or WGL. STIL or WGL pattern file converters use this format to create IG-XL-compatible files. [Scan Pattern Creation Procedure](#) shows how to create an IG-XL-compatible scan pattern file (ASCII text pattern, or .atp files). IG-XL supports scan patterns in both single and dual timing modes.

This section covers the following topics:

- | | |
|---|---|
| • | Dual Timing Mode Vector Groups |
| • | Scan and Parallel Vector Data Padding |
| • | Scan Pattern Creation Procedure |

For more information on scan patterns and ATPG, see the [TTN Scan Test Fail Translation and Diagnosis](#).

<	>
-----------------	-----------------

scanprog.3.15

<

>

SCAN Programming : Creating Scan Patterns : Dual Timing Mode Vector Groups

Dual Timing Mode Vector Groups

A scan vector is considered to be an entire vector group. That is, it must occur as a vector immediately following a vector group boundary, and the number of cycles must be a multiple of the vector group size (two for dual mode). The next vector is considered the start of a vector group. Dual mode scan vectors must have a cycle count that is divisible by two.

This affects scan vector placement. For example, in dual mode, you need a multiple of two parallel vectors before the first scan vector (this multiple can be zero). Likewise, the number of parallel vectors between scan vectors must be a multiple of two (0, 2, 4, 6, and so on).

For example, the following list shows vector numbering and grouping relationships for a valid dual mode scan pattern with four parallel vectors, then a scan vector, then two more parallel vectors, another scan vector, and finally, two more parallel vectors.

Vector number	Vector type
-----	-----
0	parallel vector
1	parallel vector
----	<-----vector group boundary
2	parallel vector
3	parallel vector
----	<-----vector group boundary
4	scan vector // Must be an even number of cycles
----	<-----vector group boundary
5	parallel vector
6	parallel vector
----	<-----vector group boundary
7	scan vector // Must be an even number of cycles

8 parallel vector

9 parallel vector

----- <-----vector group boundary

<

>

2015 Teradyne, Inc. All rights reserved.

scanprog.3.16

<	>
-----------------	-----------------

SCAN Programming : Creating Scan Patterns : Scan and Parallel Vector Data Padding

Scan and Parallel Vector Data Padding

When creating your scan patterns, you can use scan or parallel vector data padding to ensure that the number of cycles is a multiple of the vector group size. In a dual mode application, you can insert scan or parallel vectors to make an odd number of scan cycles even.

Scan pattern data padding affects how software reports scan pattern failures, specifically, the reported tester cycle number when a scan vector failure is datalogged. As a result, you should determine your scan failure reporting and diagnosis data requirements before choosing a scan pattern padding technique.

Whether you are using scan or parallel vector data padding, you can datalog the following information:

- | | |
|---|--|
| • | Pin name |
| • | Tester cycle number |
| • | Scan index number (scan vector cycle number) |

How this information is translated or interpreted for scan failure diagnosis depends on the scan pattern data padding technique.

Scan Vector Data Padding

When padding scan vectors to be modulo 2, the first cycle of a scan-in pin is padded with any drive high or low pin data, and the last cycle of a scan-out pin is padded with don't care (X) data. An implication of using scan vector padding is that the parallel vectors must be properly grouped. You may need time set switching to prevent scan chains or device clocks from being unintentionally toggled when the parallel vector is executed.

When using scan vector data padding, scan index number 0 always corresponds to the first scan chain unload shift cycle. This means that the scan index numbers often correspond directly to device scan chain location (first scan cell) for the device scan chains. When this padding technique is used, all scan chain failures can be reported in the context of scan index numbers. Despite this, you still need the tester cycle number to determine which ATPG scan pattern a given scan chain failure occurred in.

By using the scan vector padding method, you can realize the following benefits:

- Datalogged scan index numbers often correspond directly to device scan chain location (scan cell) for the longest device scan chain.
- Keeps all scan chain load and unload cycles in a scan vector.

However, you may also encounter the following limitations:

- Scan failure translation tools used to translate ATE failures to ATPG diagnosis information may not support scan index numbers.
- The DUT may not support the additional "dummy" scan cycles.

Parallel Vector Data Padding

An alternative to scan data padding is to use parallel vectors for some of the cycles that would actually perform scan chain load and unload cycles as far as the DUT is concerned.

Using this technique in dual mode is straight-forward. Simply remove one scan vector cycle and put the scan pin data into a parallel vector.

When using parallel vector padding, certain scan cycles occur outside of the scan vector. This means that scan index numbers no longer correspond directly to specific scan cells. Consequently, you need the tester cycle number to determine the scan chain failure location in the device. This requires sophisticated tester cycle to ATPG diagnosis mapping; however, it provides both scan chain location and ATPG scan pattern number information.

By using the parallel vector padding method, you can realize the following benefits:

- You can map datalogged tester cycle numbers to specific scan chain states.
- Scan failure translation tools used to translate ATE failures to ATPG diagnosis information often support tester cycle numbers.

However, be aware of the following limitations:

- Mapping tester cycle numbers to specific scan cells is a data-in.
- Splits scan chain load and unload cycles between parallel and scan vectors. Keep this in mind when using the PatternTool to review, edit, or debug test patterns.

✓ Note:

A "tester pattern" usually has many "ATPG scan patterns" (scan chain load/unload loops).

		
	2015 Teradyne, Inc. All rights reserved.	

scanprog.3.17

	
---	---

SCAN Programming : Creating Scan Patterns : Scan Pattern Creation Procedure

Scan Pattern Creation Procedure

To create a scan pattern file, follow this procedure:

1. Set up the digital instrument and pattern generator opcode timing mode for your pattern.

For SCAN-compatible patterns, you must specify the HSD1000 or UltraPin800 board. You can use either single or dual timing mode. For example:

```
digital_inst = hsdm;  
opcode_mode = dual;
```

2. Specify the workbook that contains the pin map for this pattern.

```
pinmap_workbook = "my_job.xls";
```

3. Import the time set used in your pattern file.

```
import tset ts_shift, ts_noshift;
```

4. Set up your SCAN type statement.

The pattern's compiler control statement indicates if you are using SCAN-compatible or parallel vectors. If you are using SCAN hardware, it also indicates the SCAN type you are using. Once you have chosen the correct SCAN type for your application in [Selecting SCAN Type](#), you can use the `scan_type` statement to set it up.

For instance, to set up a X2 SCAN type:

```
scan_type = X2;
```

5. List your scan pins.

Use the `scan_pins` statement to identify the scan patterns in your source code. For example, use the following syntax to list your scan pins, where `pin0` is assigned to Ch8, `pin1` to Ch9, `pin2` to Ch10, and `pin3` to Ch11:

```
scan_pins = {
    pin0,
    pin1,
    pin2,
    pin3
}
```

6. Specify the memory to which the scan pattern is loaded.

You must indicate whether you are loading patterns to the VM or the SRM. For SCAN-compatible vectors, you must use the VM. To do this, use the following statement:

```
vm_vector <vector_name> ( <time_set>, <pin_name>, <pin_name>, ... ) {
    <scan_vector_data>;
    <parallel_vector_data>;
    ...
}
```

For example:

```
vm_vector scan_test1 ( $tset, A0, A1, A2, A3 ) {
    // Scan and parallel vectors listed here
}
```

7. Specify the pin data for non-scan pins during scan vector execution.

By default, the pin data for non-scan pins during scan vector execution is set to the pin data of the parallel vector immediately before the scan vector. However, if you require a specific data state for non-scan pins, then you can use the `scan_setup` statement to specify the pin data for those pins.

For example:

```
scan_setup
> ts_shift 0X0X 11XX;
```

☒ Note:

You can include parallel vectors throughout your scan test pattern without using the `scan_setup` statement.

8. [Create your scan vectors.](#)

Use the [scan](#) statement to define the number of cycles for which scan data should be used, the scan pin type (scan-in or scan-out), the scan pins to be programmed, their radices, and the data for the scan pins.

`scan 10`

```
( si  pin0:S  > 0110101110 )
( si  pin2:S  > 1101010110 )
( so  pin1:S  > LLHXLHLHHL )
( so  pin3:S  > LHLLHXHLLH );
```

For more information, see [SCAN Syntax Reference](#).

Example

The following example shows a simple scan pattern file.

```
digital_inst = hsdm;
```

```
pinmap_workbook = "..\Scan_Validation_1.1.xls";
```

```
opcode_mode = dual;
```

```
import tset sc_rst;
```

```
scan_type = x2;
```

```
// A0, A1, A2, A3, A4, A5, A6 are scan pins.
```

```
// A7, A8, A9 are parallel pins.
```

```
// -----
```

```
scan_pins = {
```

```
    A0,
```

```
    A1,
```

```
    A2,
```

```
    A3,
```

```
    A4,
```

```
    A5,
```

```
    A6,
```

```
}
```

```
vm_vector scan_mod0( $tset,  A0  A1  A2  A3  A4  A5  A6  A7  A8  A9)
```

```
{
```

```
// Two parallel vectors
```

```
> sc_rst 0 L M X 1 H V - 0 1;
```

```
> sc_rst 1 H V - 0 L M X 1 0;
```

```

// Scan setup vector.
// A7, A8, A9 pin data will repeat this vector while in scan.
scan_setup
    > sc_rst  1  0  X  X  X  X  X  L  X  H;

// Execute 40 scan cycles. This syntax uses repeated strings
// (for instance, "[10-0101]") to repeat four bits of data 10 times.
scan 40
( si A0 > [10-0101] )
( so A1 > [10-LHLH] )
( si A2 > [10-1001] )
( so A3 > [10-HLLH] )
( si A4 > [10-0100] )
( so A5 > [10-LHHH] )
( si A6 > [10-1001] );

// Two parallel vectors
    > sc_rst  1  1  0  0  0  0  0  0  1  1;
    > sc_rst  L  H  1  V  0  0  0  0  M  V;

// Execute 10 scan cycles.
scan 10
( si A0 > 0101101010 )
( so A1 > HLHLXXLHLH )
( si A2 > 1010100101 )
( so A3 > XLHLLHHLX )
( si A4 > 0101010101 )
( so A5 > LHLHLHLHLH )
( si A6 > 0101010101 );

// Execute parallel vectors to meet the 128 vector limit in a pattern file
// running in dual mode.

    > sc_rst  1  1  0  0  0  0  0  0  X  X;
    > sc_rst  1  1  0  0  0  0  0  0  L  H;
    > sc_rst  1  1  0  0  0  0  0  0  1  0;
    > sc_rst  1  1  0  0  0  0  0  0  M  V;
    > sc_rst  1  1  0  0  0  0  0  0  -  -;

```

```
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
```

```
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;  
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
```

```
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;  
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
```

```
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;  
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
```

```

> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;

```

```

> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;

```

```

> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;

```

```

> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;

```

```
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
```

```
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
```

```
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
```

```
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
> sc_rst 1 1 0 0 0 0 0 0 0 - -;
> sc_rst 1 1 0 0 0 0 0 0 0 X X;
> sc_rst 1 1 0 0 0 0 0 0 0 L H;
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;
> sc_rst 1 1 0 0 0 0 0 0 0 M V;
```

```
> sc_rst 1 1 0 0 0 0 0 0 0 - -;  
  
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;  
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;  
  
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
> sc_rst 1 1 0 0 0 0 0 0 0 - -;  
> sc_rst 1 1 0 0 0 0 0 0 0 X X;  
> sc_rst 1 1 0 0 0 0 0 0 0 L H;  
> sc_rst 1 1 0 0 0 0 0 0 0 1 0;  
> sc_rst 1 1 0 0 0 0 0 0 0 M V;  
halt > sc_rst 1 1 0 0 0 0 0 0 0 - -;  
}
```

<

>

scanprog.3.18

	
---	---

SCAN Programming : Compiling Scan Patterns

Compiling Scan Patterns

Use the [Pattern Compiler GUI](#) or the [command line interface](#) (apc) to compile your scan patterns.

Type `apc -h` for a list of pattern compiler options.

Scan test patterns can be compiled as either [parallel](#) or [SCAN-compatible](#) patterns. Scan patterns are created using the scan microcode in your pattern file.

Compiling Parallel Patterns

When you compile scan test patterns as [parallel patterns](#) (compiled using `-scan_type parallel` compiler option), the following applies:

- [Scan pin data is treated no differently than regular parallel pin vector data.](#)
- [SCAN hardware is not used during pattern execution.](#)
- [VM pattern memory cannot be shared across channels.](#)
- [Pattern pin data requires three bits per cycle.](#)
- [The maximum scan pattern depth is limited by the maximum VM depth \(128Mcycles\).](#)
- [This mode does not require a SCAN hardware license.](#)

You can also compile parallel patterns by including the following syntax directly into the pattern file:

```
scan_type = parallel;
```

 Note:

If you are doing scan testing, but are not using the SCAN hardware packing in your application, Teradyne recommends using the scan pattern syntax and the parallel compiler option, rather than using a pattern translation process, to create parallel vectors. This approach allows you to use the scan hardware packing in the future if your scan depth requirements change. Moreover, it retains scan chain information in the pattern file, which can be used by the pattern debug tool and other scan diagnostic tools.

Compiling SCAN-Compatible Patterns

When you compile UltraFLEX test patterns as SCAN-compatible patterns (the `-scan_type parallel` compiler option is not used), the following applies:

- | | |
|---|--|
| • | Scan pin data uses HSD1000, UltraPin800, or UltraPin1600 SCAN hardware. |
| • | VM pattern memory can be pooled across several digital channels. |
| • | Scan-in pattern pin data requires only one bit per cycle. |
| • | Scan-out pattern pin data requires two bits per cycle. (You can use one bit with only two pin data states for two-bit scan chains.) |
| • | The maximum scan pattern depth is limited by the SCAN type defined in the pattern file (<code>scan_type = X2 X3 X4 X6 X8 X12 Extended</code>). |

For information on scan chain configurations, including the maximum scan chain depth and maximum number of scan chains for each SCAN type, see [Three-bit Scan Chain Configurations per Digital Board](#) and [Two-bit Scan Chain Configurations per Digital Board](#). This increase in maximum pattern depth can be achieved because SCAN-compiled patterns can be stored more efficiently than parallel-compiled scan vectors.

scanprog.3.19

<

>

SCAN Programming : Debugging Using PatternTool

Debugging Using PatternTool

Once you create and compile a scan pattern, you can review and edit that pattern using PatternTool. By default, PatternTool does not display the rows for the scan chains:

TSet	Command	Label	Vector	d i r	e	portA	portB	t d i	t d o	t c k	t m s
tset_scan			28	X	X	XXXXXXXX	XXXXXXXX	1	L	1	1
tset_scan			29	X	X	XXXXXXXX	XXXXXXXX	1	X	1	1
tset_scan			30	X	X	XXXXXXXX	XXXXXXXX	1	X	1	0
tset_scan			31	X	X	XXXXXXXX	XXXXXXXX	1	X	1	1
tset_scan			32	X	X	XXXXXXXX	XXXXXXXX	1	X	1	0
tset_scan			33	X	X	XXXXXXXX	XXXXXXXX	1	X	1	0
tset_scan	scan 10 scan_setup		34	X	X	XXXXXXXX	XXXXXXXX	0	L	1	0
tset_scan			35	X	X	XXXXXXXX	XXXXXXXX	1	L	1	1
tset_scan			36	X	X	XXXXXXXX	XXXXXXXX	1	X	1	1

PatternTool without Scan Chains

 Scan chains

 To display the scan chains, click the Scan Chains button. Chains are displayed as follows:

TSet	Command	Label	Vector	d	i	o	portA	portB	t	d	t	t	t
tset_scan			31	X	X		XXXXXXXX	XXXXXXXX	1	X	1	1	
tset_scan			32	X	X		XXXXXXXX	XXXXXXXX	1	X	1	0	
tset_scan			33	X	X		XXXXXXXX	XXXXXXXX	1	X	1	0	
tset_scan	scan 10 scan_setup		34	X	X		XXXXXXXX	XXXXXXXX	0	L	1	0	
			34:0						1	L			
			34:1						0	H			
			34:2						0	L			
			34:3						1	L			
			34:4						1	H			
			34:5						0	H			
			34:6						1	L			
			34:7						1	H			
			34:8						0	H			
			34:9						0	L			
tset_scan			35	X	X		XXXXXXXX	XXXXXXXX	1	L	1	1	
tset_scan			36	X	X		XXXXXXXX	XXXXXXXX	1	X	1	1	
tset_scan			37	X	X		XXXXXXXX	XXXXXXXX	1	X	1	0	
tset_scan			38	X	X		XXXXXXXX	XXXXXXXX	1	X	1	1	
tset_scan			39	X	X		XXXXXXXX	XXXXXXXX	1	X	1	1	

Scan Chains in PatternTool

The full set of editing operations is available on both scan and parallel vectors. You can cut, copy, and paste vectors.

For more information, see [Viewing and Editing Scan Chains](#) in the PatternTool section.

<

>

2015 Teradyne, Inc. All rights reserved.

scanprog.3.20

<	>
-----------------	-----------------

SCAN Programming : Using Deep Scan Capture

Using Deep Scan Capture

Deep scan capture is a programming technique used to capture large amounts of PatGen information used during scan. This technique uses information from the central capture memory (CMEM) combined with the pass/fail data from the pin CMEM to capture up to 256K locations of scan data. This section is divided into the following topics:

- | | |
|--|--|
| | • Deep Scan Capture Hardware |
| | • Setting up Deep Scan Capture |
| | • Reading Deep Scan Capture Data |
| | • Deep Scan Capture VBT Code Example |

<	>
-----------------	-----------------

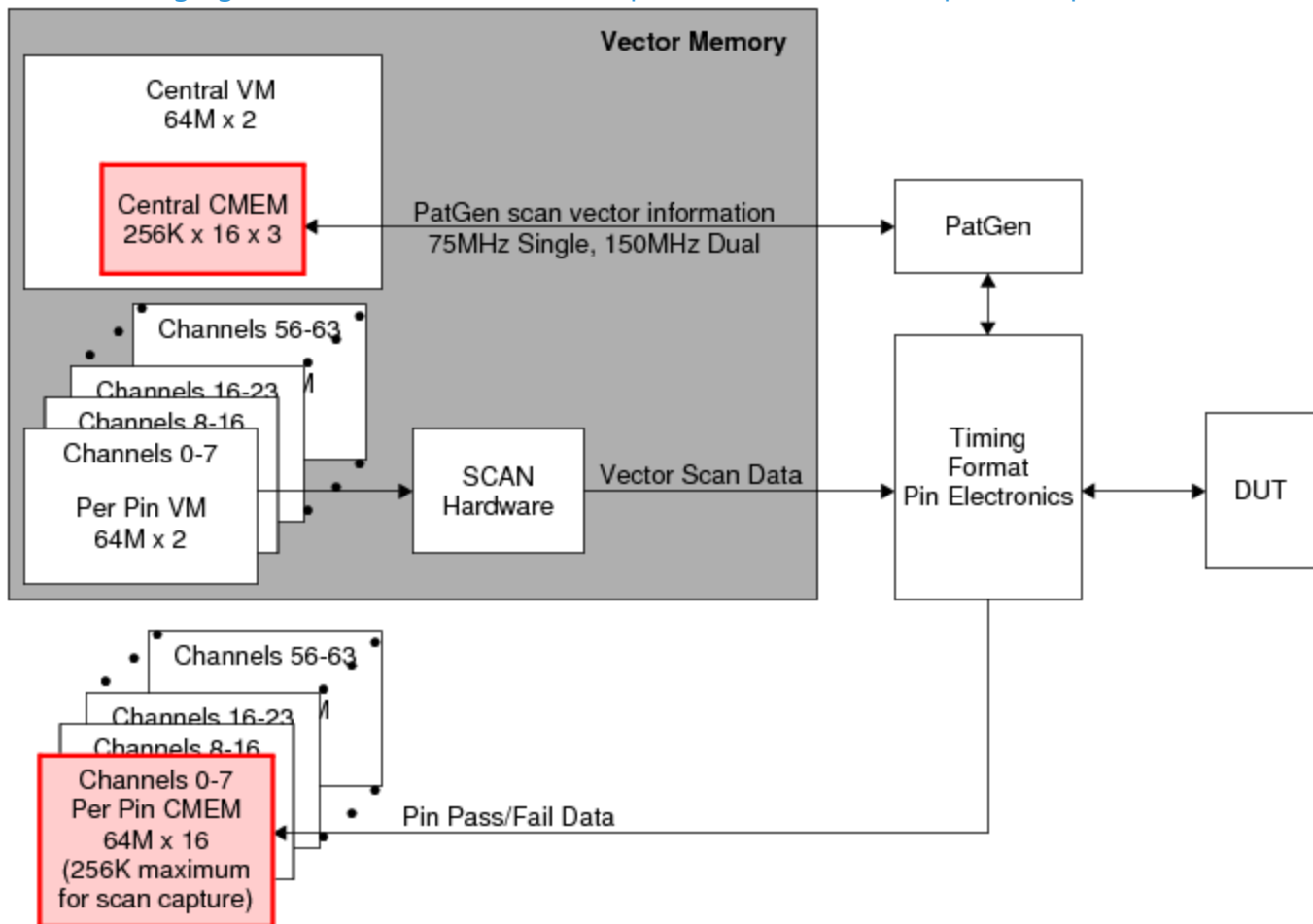
scanprog.3.21



SCAN Programming : Using Deep Scan Capture : Deep Scan Capture Hardware

Deep Scan Capture Hardware

The following figure shows the hardware components that allow deep scan capture.



HSD1000 Digital Board Deep Scan Capture Hardware

Central CMEM and VM

The central CMEM handles the setup and readback of scan fail data information. This central CMEM is located in the central vector memory (VM) that supports the digital instrument's pattern generator. This VM is 64Mvectors by 2, where by 2 represents dual mode operation when both

64M VMs are running together. The central VM reserves a 256K x 16 x 3 memory space for central CMEM, which can store 256K major cycles in three 16-bit wide fields in single mode and 512K locations in dual mode.

Since central CMEM uses the same memory as the central VM, the VM has to source and capture simultaneously causing a bandwidth bottleneck. As a result, deep scan capture is guaranteed at vector rates only up to the maximum values listed in the table below:

Deep Scan Capture Maximum Vector Rates

Digital Instrument	Maximum Vector Rate (Vectors/Second)	
	Single Mode	Dual Mode
HSD1000	75M	150M
UltraPin800	75M	150M
UltraPin1600	200M	400M

If the bandwidth is exceeded, IG-XL returns an error that prevents you from running the job at that data rate.

HSDMPI:1465 : Invalid period (<period>) specified for timeset <timeset name> when capturing central CMEM data. Period must be greater than <maximum period> in timing mode <single|dual>.

Note that you can disable this error and run at a higher data rate, but you may still see an overflow alarm. By disabling this error, you may be able to run at faster data rates depending on the application’s size and number of modules, density of scan fails, and the amount of VM data sent to the PatGen. Teradyne does not guarantee that this feature will work correctly, however, if you disable the IG-XL error and program your Time Sets sheet to exceed the maximum data rates listed above.

Central CMEM Fields

Deep scan capture is mainly used for storing SCAN failures, but you can also use the central CMEM to store most PatGen information. This includes absolute cycle count, module cycle count, sync cycle count, VM offset, VM pattern, scan index, and loop/repeat count. You can read back these data storage fields and use them for any application. The central CMEM cannot store as much information as the HRAM. For instance, central CMEM does not store which pin is failing. When triggered, it stores only the selected PatGen information.

The following table lists the number of 16-bit fields and HSD1000 boards required to store these data fields. Make sure that you have enough boards to store the PatGen information that the DUT requires. Remember that each HSD1000 board only has three 16-bit fields in the central CMEM for both single and dual modes.

HSD1000 CMEM Central Data Storage Field Requirements

Data Storage Field	Number of 16-bit Fields Required	Number of HSD1000 PatGens Required
Absolute cycle count	3	1
Module cycle count	3	1
Sync cycle count	3	1
VM offset	2	1
VM pattern	2	1
Scan Index	6	2
Loop/repeat count	2	1

Pin CMEM

The pin CMEM can store either pass/fail or device pin data information and has 265K of memory behind each channel. Typically, scan applications store only pass/fail data. When data from the pin CMEM is combined with the central CMEM’s PatGen scan vector information, you have the all the required information needed to understand the stored cycle. The central CMEM and pin CMEM are always synchronized through the use of a common trigger signal.

Pin VM

The HSD1000 board pin VM supports 64 channels, sliced into groups of eight. Each pin VM channel group has two ports to support dual mode operations with 64Mvectors of memory behind each port. In single mode, only one port is active.

The pin VM stores the pattern’s vector data (0, 1, L, H, X, V, M). Note that you cannot capture this data during deep scan capture. For each cycle, this VM sends three bits of data to represent the pin data. It also supports scan data, which is typically compressed in VM. The SCAN hardware allows you to take the eight channels and three bits per channel and combine them to get better data compression in VM.

For information on how to compress scan data effectively in VM, see SCAN Types.

scanprog.3.22

<	>
---	---

SCAN Programming : Using Deep Scan Capture : Setting up Deep Scan Capture

Setting up Deep Scan Capture

To set up deep scan capture, use the following methods in a VBT module:

- `TheHdw.Digital.CMEM.SetCaptureConfig`: Sets the number of CMEM cycles to be captured and the type of capture data.

Note that you can also read back the capture configuration for the CMEM using `TheHdw.Digital.CMEM.GetCaptureConfig`.

On the HSD1000, UltraPin800, and UltraPin1600, deep scan capture is only supported in single 1X and dual 1X modes. Quad mode is not supported (UltraPin1600 only).

- `TheHdw.Digital.CMEM.CentralFields`: Sets up which data storage fields to capture in central CMEM

For an example that uses these methods, see [Deep Scan Capture VBT Code Example](#).

<	>
---	---

scanprog.3.23

<	>
---	---

SCAN Programming : Using Deep Scan Capture : Reading Deep Scan Capture Data

Reading Deep Scan Capture Data

To read deep scan capture data, use the following methods in a VBT module:

- [TheHdw.Digital.CMEM.PatGenInfo](#): Gets the pattern generator information stored in the CMEM.
- [TheHdw.Digital.CMEM.PatternName](#): Gets the pattern module names stored in the CMEM.
- [TheHdw.Digital.Pins.CMEM.StoredCycleData](#): Returns the per-site fail cycle scan data.

For an example that uses these methods, see [Deep Scan Capture VBT Code Example](#).

<	>
---	---

scanprog.3.24

	
---	---

SCAN Programming : [Using Deep Scan Capture](#) : Deep Scan Capture VBT Code Example

Deep Scan Capture VBT Code Example

The following VBT code module shows how to program deep scan capture and read back the scan data.

Option Explicit

```
Dim isCMEMDataValid As Boolean ' Global flag
```

```
' This function is called right before an alarm is reported.
```

```
' The alarmList is a tab delimited string of alarm error messages.
```

```
Public Function OnAlarmOccurred(alarmList As String)
```

```
    On Error GoTo errHandler
```

```
    Dim i As Long
```

```
    Dim alarmArray() As String
```

```
' The string is a tab delimited list of alarm error messages.
```

```
    alarmArray = Split(alarmList, vbTab)
```

```
' This section loops through all the alarms.
```

```
    For i = LBound(alarmArray) To UBound(alarmArray)
```

```
        ' Check for FIFO Overflow Alarm.
```

```
        If InStr(1, alarmArray(i), "HSDMPI:0164") Then
```

```
            isCMEMDataValid = False
```

```
        End If
```

Next i

Exit Function

errHandler:

HandleExecIPError "OnAlarmOccurred"

End Function

' This function sets up scan data capture at the Start of Body.

Public Function StartOfBodyIPF(argc As Integer, argv() As String)

' Set to True here and False in OnAlarmOccurred.

isCMEMDataValid = True

' Capture all failures (full 256K single or 512K dual).

Call TheHdw.Digital.CMEM.SetCaptureConfig(-1, CmemCaptFail)

TheHdw.Digital.CMEM.CentralFields = tlCMEMAbsoluteCycle + tlCMEMPatternName + _
tlCMEMVMVectorOffset + tlCMEMScanIndex

End Function

' This function turns off central capture at the End of Body.

Public Function EndOfBodyIPF(argc As Integer, argv() As String)

' Use this statement to turn off the Pin CMEM capture.

Call TheHdw.Digital.CMEM.SetCaptureConfig(0, CmemCaptNone)

' Use this statement to turn off the Central CMEM capture.

TheHdw.Digital.CMEM.CentralFields = tlCMEMNone

End Function

' Support function for FailingPins function

Public Function DoubleToBinaryString(Data As Double) As String

' The results stored here as a string that represents which pins have failed.

Dim bitStream As String

bitStream = vbNullString

Dim k As Integer

Dim theData As Double ' Used to compare.

Dim temp As Double ' Added I

temp = Data

' Loop and keep shifting the data down.

For k = 0 To 31

```

theData = Floor(temp / 2)
' Data is even if theData * 2.0 is equal to Data.
If theData * 2 = temp Then
    bitStream = "0" + bitStream
Else
    bitStream = "1" + bitStream
End If
temp = Floor(temp / 2) ' Shift down to next bit.
Next k

```

```

DoubleToBinaryString = bitStream
End Function

```

```

' Support function for printing failing pins.
' Pass in failing pins as a double, the pins as an array of strings, and the offset.
' The offset is for greater than 32 pins.
' The pinMask is a double with the max value of 32 bits.
' The offset is 0 for 1 to 32 pins, 1 for 33 to 64 pins, 2 for 65 to 96, and so on.
Public Function FailingPins(pinMask As Double, pins() As String, offset As Long) _
    As String
    Dim i As Long
    Dim strMask As String
    Dim tempStr As String

    strMask = DoubleToBinaryString(pinMask)
    FailingPins = vbNullString
    For i = 0 To 31
        tempStr = Mid$(strMask, 32 - i, 1)
        If (tempStr = "1") Then
            FailingPins = FailingPins + pins(i + offset * 32) + ","
        End If
    Next i

    ' Get rid of trailing ","
    If Len(FailingPins) > 0 Then
        FailingPins = Left$(FailingPins, Len(FailingPins) - 1)
    End If

```

End Function

' argv(0) is the name of pins for which to retrieve data.

Public Function PostTestIPF(argc As Long, argv() As String)

On Error GoTo errHandler

If argc <> 1 Then

TheExec.Datalog.WriteComment ("NEED TO PASS PINLIST")

Exit Function

End If

' If all sites passed, then just return.

If TheHdw.Digital.Patgen.PatternBurstPassed Then Exit Function

' If error occurred, then exit.

If isCMEMDataValid = False Then Exit Function

' Get the number of capturedCycles.

Dim capturedCycles As Long

capturedCycles = TheHdw.Digital.CMEM.capturedCycles

TheExec.Datalog.WriteComment ("Number of CapturedCycles: " & capturedCycles)

' Read the failing pins for all sites, compressing the failures.

Dim sitePinData As New SiteVariant

Dim siteIndexData As New SiteVariant

Dim indexArr() As Long ' Used in site loop

Dim pinDataArr() As Double ' Used in site loop

Call TheHdw.Digital.pins(argv(0)).CMEM.StoredCycleData(siteIndexData, _
sitePinData, capturedCycles, True)

' Read back the last index where data was captured for each pattern.

Dim patternNames() As String, lastFailPerPattern() As Long

Call TheHdw.Digital.CMEM.PatternName(lastFailPerPattern, patternNames)

' Read back the central data.

Dim cycleData() As Double ' Data type is Double because cycle count is 40 bits.

Dim scanIndexData() As Double

Dim vectorData() As Double

cycleData = TheHdw.Digital.CMEM.PatGenInfo(tlCMEMAbsoluteCycle, capturedCycles)

scanIndexData = TheHdw.Digital.CMEM.PatGenInfo(tlCMEMScanIndex, capturedCycles)

```
vectorData = TheHdw.Digital.CMEM.PatGenInfo(tlCMEMVMVectorOffset, _  
    capturedCycles)
```

```
' Decompose the pins to be used in reading out the failures.
```

```
Dim nPins As Long
```

```
Dim pins() As String
```

```
Call TheExec.DataManager.DecomposePinList(argv(0), pins, nPins)
```

```
' Use the module pattern name as the name of the scan fail file.
```

```
Dim patName As String
```

```
Call TheHdw.Digital.Patgen.ReadLastStart(patName, 0, 0)
```

```
If InStr(UCase(patName), ".PAT") Then ' If a .pat file, get rid of the '.PAT'.
```

```
    patName = Left(patName, Len(patName) - 4)
```

```
End If
```

```
' File support (replace with dll)
```

```
Dim fileNum As Long
```

```
fileNum = FreeFile
```

```
Dim oFileName As String
```

```
Dim ScanFileDir As String
```

```
ScanFileDir = "C:\Temp\"
```

```
' Variables used in site loop
```

```
Dim site As Variant
```

```
Dim deviceNumber As Long
```

```
Dim patIdx As Long
```

```
Dim pinStr As String
```

```
Dim i As Long, k As Long
```

```
Dim bigPinList As Integer ' Used when pinlist is greater than 32.
```

```
For Each site In TheExec.Sites
```

```
    ' Move siteVariant to normal array of Long/Double
```

```
    pinDataArr = sitePinData
```

```
    indexArr = siteIndexData
```

```
    ' bigPinList will equal 1 for <32 pins; 2 for 64 to 33 pins; and so on.
```

```
    bigPinList = UBound(pinDataArr) \ UBound(indexArr)
```

```
    ' Create a new scan file for each site and each device.
```

```
    deviceNumber = TheExec.Datalog.setup.LotSetup.deviceNumber + site
```

```
oFileName = ScanFileDir & patName & "_" & deviceNumber & ".txt"
```

```
Open oFileName For Output As #fileNum
```

```
patIdx = 0
```

```
For i = LBound(indexArr) To UBound(indexArr)
```

```
    pinStr = vbNullString
```

```
    For k = 0 To bigPinList - 1
```

```
        pinStr = FailingPins(pinDataArr(i * bigPinList + k), pins, k) & _  
        "," & pinStr
```

```
    Next k
```

```
    ' Get rid of trailing ","
```

```
    If Len(pinStr) > 0 Then
```

```
        pinStr = Left$(pinStr, Len(pinStr) - 1)
```

```
    Else
```

```
        GoTo nexti ' Do not print if there are no failing pins.
```

```
    End If
```

```
    If patIdx > UBound(patternNames) Then
```

```
        ' Raise error. This should never happen.
```

```
    End If
```

```
    If i > lastFailPerPattern(patIdx) Then
```

```
        ' Go to the next pattern in the list.
```

```
        patIdx = patIdx + 1
```

```
    End If
```

```
    ' patternname vectoroffset scanindex failingpins
```

```
    Print #fileNum, patternNames(patIdx) & " " & vectorData(indexArr(i)) & _  
    " " & scanIndexData(indexArr(i)) & " " & pinStr
```

```
    ' absolutecycleforallpatterns failingpins
```

```
    Print #fileNum, cycleData(indexArr(i)) & " " & pinStr
```

```
nexti:
```

```
    Next i
```

```
    Close #fileNum
```

```
Next site
```

```
Exit Function
```

```
errHandler:
```

```
    Call TheExec.LogErrorMessage("Test " & Err.Description & ", Instance: " & _  
    TheExec.DataManager.instanceName)
```

```
    Close #fileNum
```

```
End Function
```


' Disables the deep scan capture timing error.
Public Function DisableTimingError()

 TheExec.Error.Behavior("HSDM:XXXX") = tlErrorIgnore

End Function

<		>	
	2015 Teradyne, Inc. All rights reserved.		

scansyntax.4.1

<	>
----------------------	----------------------

SCAN Syntax Reference

SCAN Syntax Reference

This section provides a reference for all SCAN pattern file syntax and contains the following topics:

- | | |
|---|----------------------------|
| • | scan |
| • | scan_pins |
| • | scan_setup |
| • | scan_type |

<	>
----------------------	----------------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scansyntax.4.2

<

>

SCAN Syntax Reference : scan

scan

The scan microcode identifies a scan vector in the pattern text file. This type of vector uses a special format that includes the number of cycles for which scan data should be used, the scan pins to be programmed, their radices, the scan pin type (scan-in or scan-out), and data for the scan pins. By convention, a scan vector has a single vector number even though it generates multiple tester cycles.

Note that the value after the keyword scan in a scan vector is the vector cycle count, not the PatGen cycle count.

Format

```
scan <cycles>
  ( <scan_pin_type> <scan_pin>[:<radix>]  > <scan_data> )
  [ ( <scan_pin_type> <scan_pin>[:<radix>]  > <scan_data> ) ]
  ...
;
```

Parameters

cycles	The number of scan cycles to execute for this vector. IG-XL does not impose a limit on the number of scan cycles.
scan_pin_type	The scan pin type, which can be either si for scan-in pins or so for scan-out pins.
scan_pin	The name of a pin, as specified in the scan_pins statement.
radix	An optional parameter that can be either S for symbolic, X or H for hexadecimal. Default is symbolic.

scan_data

The scan data for a scan-in or scan-out pin that contains the sequence of scan data that are actually sent to the named pin.
Each line of scan data has a limit of 10032 characters. IG-XL returns a compiler error if the number of characters exceeds this limit. To work around this limit, you can break up a long line of scan data into multiple lines or use repeated strings (see Using Repeated Strings). When breaking up long lines of scan data, you do not need to include a line continuation delimiter. The pattern compiler assumes that it is on the same line of scan data until it reaches a semi-colon, which is the end of line delimiter.
For more information, see Scan Data.

Usage

Scan Data

Every scan pin in a scan pattern must be programmed in a scan vector. The pattern compiler returns an error if you omit data for scan pins in individual scan vectors.

Scan data can be programmed only in `vm_vector` blocks.

Scan data is allowed in both single and dual timing mode.

In dual timing mode, scan data must start in the first vector of a vector pair and end at the second vector of a vector pair, after packing; that is, scan data must start and end at a major vector boundary. See Dual Timing Mode Vector Groups.

In the sequence of scan data bits, the leftmost datum (MSB) is presented to the device first or compared first.

The scan pin data can be in either symbolic or hexadecimal format. If the pin data type is not specified, the default is symbolic.

Symbolic Format

The following tables show how scan data is encoded in symbolic format.

The number of symbolic scan characters for a pin must match exactly the number of scan cycles specified for that scan vector.

White space can be inserted anywhere in the scan data to improve readability. White space must be used if the number of cycles in the scan vector is greater than 1024, since the pattern compiler does not handle single strings with more than 1024 characters.

Symbolic Format for Scan-in Pin Data

Scan Input Data	Two-Bit and Three-Bit Scan Chains
0	Drive low

1	Drive high
---	------------

Symbolic Format for Scan-out Pin Data

Scan Output Data	Three-Bit Scan Chains	Two-Bit Scan Chains
L	Expect low	Expect low
H	Expect high	Expect high
X	Don't care	Illegal

Hexadecimal Format

Hexadecimal format is more compact than symbolic format, but is harder to read. It represents four bits of scan data in a single character, with the MSB (4-bit) being presented to the DUT first. The following tables show how scan data is encoded in hexadecimal format. Some of the LSBs in the last hexadecimal character may be ignored if not needed for the number of cycles in the scan vector. Such extra bits must be 0. It is an error to have too few characters for the number of scan cycles, or to have too many characters. As with symbolic data, white space can be used anywhere and is required if there are more than 1024 hexadecimal characters.

Hexadecimal Format for Scan-in Pin Data

Scan Input Data	Two-Bit and Three-Bit Scan Chains
0	Drive low
1	Drive high

Hexadecimal Format for Scan-out Pin Data

Scan Data	Two-Bit Scan Chains	Three-Bit Scan Chains
0	Expect low	Illegal
1	Expect high	Illegal
00	Illegal	Expect low
01	Illegal	Expect high
10	Illegal	Illegal

Using Repeated Strings

You can use a shorthand method for specifying repeated strings in scan data. This method can be mixed with enumerated data. You can use this shorthand for either symbolic or hexadecimal data. It is not displayed in the pattern tools and is not preserved if the pattern binary is reverse compiled. Brackets cannot be nested.

The syntax for this shorthand method is as follows:

[NNN-SSS]

where NNN is the decimal number of repetitions, - is a separator, and SSS is the string to be repeated. This SSS string cannot have more than 1024 characters.

For example, [5-01] is interpreted as 0101010101.

Example

The following example shows a scan data vector that runs for 10 cycles for two scan pins using the symbolic format.

```
vm_vector IOTEST ( $tset, pin0, pin 1 ){  
...  
    ( si  pin0:S  >  0100010101 )  
    ( so  pin1:S  >  HHLLXHLHLL );  
...  
}
```

scansyntax.4.3



SCAN Syntax Reference : scan_pins

scan_pins

This statement indicates which pins are used as scan pins. Scan patterns can be identified by the presence of a scan_pins statement in the pattern source code.

Format

```
scan_pins = {  
    <scan_pin>,  
    <scan_pin>,...  
}
```

Parameters

scan_pin is the name of a single pin in the pin map (pin groups are not accepted). This can be a scan-in or scan-out pin.

Usage

For information on how to assign scan-in and scan-out pins, see [Even-Odd SCAN Channel Rule](#).

All pins in a scan_pins statement must have scan vector data defined in each scan vector statement. Omitting scan pins in a scan vector produces a compiler error.

[A scan pin cannot be a digital channel number.](#)

The scan_pins statement does not accept groups of pins; each scan pin must be listed individually. Although pin group names (including groups containing scan pins) can be used in the pin list that is at the beginning of the vector statement (preceding the pattern vectors), pin group names cannot be used in the scan_pins statement.

It is an error if pins in the scan_pins statement do not also appear in the vector pinlist. Scan pins must appear somewhere in the vector pinlist, either individually or as part of a pingroup.

All scan patterns used in a single burst must have identical scan_pins statements or an error is given at pattern load time. This is required because the SCAN hardware cannot be reconfigured while a pattern burst is running.

This statement is not allowed in quad timing mode (opcode_mode = quad), which is used on the UltraPin1600.

Example

The following example shows two pins, where pin0 is assigned to Ch8 and pin1 to Ch9:

```
scan_pins = {  
    pin0,  
    pin1  
}
```

< >	
	2015 Teradyne, Inc. All rights reserved.

scansyntax.4.4

<	>
-----------------	-----------------

SCAN Syntax Reference : scan_setup

scan_setup

The scan_setup statement specifies parallel vector data for pins in the vector pin list during scan. Once established, this parallel vector state is repeated during the next scan vector, but it does not generate a tester cycle. A scan_setup vector looks like a normal parallel vector with scan_setup as the microcode, except that the list of other microcodes is restricted.

The scan_setup vector can contain the following:

- Any parallel channel data codes on parallel pins
- Any tset name or number
- Any microcode (auxiliary PatGen) commands, subject to timing mode restrictions
- An STV microcode to support capturing scan data into capture memory

Format

scan_setup

> <parallel_vector_setup>

Parameters

The parallel_vector_setup parameter is the standard format for parallel vector pin data programming.

Usage

Parallel digital test vectors are the standard pattern data vectors that do not use the scan microcode. During a scan test, parallel vectors are usually used at the beginning of the scan test pattern and interspersed between scan vectors. Generally speaking, scan pattern data is shifted into

and out of device scan chains during the scan vectors, and the parallel vectors are used to stop scan shifting and exercise the internal logic of the device.

A scan_setup vector can appear anywhere in the pattern, and the state applies to all future scan vectors until the next scan_setup statement.

A scan_setup vector is optional. If you do not include it in your pattern file, the runtime repeat (-) pin state is used as parallel data for pins in the vector pin list during scanning. For pin setups that do not support the runtime repeat character (-), 0 and X is used for input and output pins, respectively.

The datacodes specified in a scan_setup vector for the pins used in subsequent scan vectors are ignored. Labels cannot be used on scan_setup vectors. The pattern compiler gives an error in this case.

Examples

The following example shows the setup for several non-scan pins.

```
scan_setup
> ts_shift  0X0X  11XX;
```

<

>

2015 Teradyne, Inc. All rights reserved.

scansyntax.4.5

<

>

SCAN Syntax Reference : scan_type

scan_type

This compiler control statement indicates whether scan vectors should be compiled for SCAN hardware or expanded into parallel vectors.

Format

scan_type = <scan_type_choice>;

Parameters

scan_type_choice is the scan type selected for your scan application. It can be one of the following options:

Three-Bit Scan Chain Options

SCAN Type	Meaning
X2	Selects X2 SCAN type
X4	Selects X4 SCAN type
X8	Selects X8 SCAN type
Extended	Selects Extended SCAN type

Two-Bit Scan Chain Options

SCAN Type	Meaning
X3	Selects X3 SCAN type
X6	Selects X6 SCAN type

X12	Selects X12 SCAN type
X24	Selects X24 SCAN type

Non-SCAN Option

SCAN Type	Meaning
Parallel	Expands scan vectors into parallel vectors

Usage

For more information on SCAN types and how to select them for your application, see [SCAN Types](#) and [Selecting SCAN Type](#). See also [scan_type](#) in the Pattern Language section.

Example

The following example shows an X4 SCAN type
`scan_type = X4;`

<

>

2015 Teradyne, Inc. All rights reserved.

scantheory.2.01

<

>

SCAN Theory of Operation

SCAN Theory of Operation

This section provides an introduction to scan testing techniques. It also describes the HSD1000, UltraPin800, and UltraPin1600 SCAN hardware and software used to conduct scan testing and several important scan concepts.

The following topics are available:

- Internal Scan Testing
- Boundary Scan Testing
- SCAN Functional Flow Chart
- SCAN Hardware and Software
- SCAN Regions
- SCAN Vector Memory
- SCAN Types

For information on SCAN performance specifications and standards, see:

- HSD1000 Specifications

- [UltraPin800 Specifications](#)

- [UltraPin1600 Specifications](#)

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scantheory.2.02

	
---	---

SCAN Theory of Operation : Internal Scan Testing

Internal Scan Testing

The most popular design-for-test (DFT) technique is the use of internal scan. Internal scan design and test techniques are used because they provide the greatest potential for high test coverage. The main idea behind this technique is to modify existing flip-flops and/or latches in the design to support serial shifting of digital data, in addition to their normal "mission-mode" design functions. Serial shifting of digital data throughout the device is supported by connecting these elements into long shift registers that are referred to as scan chains.

A digital sequential circuit is composed of combinational and sequential logic elements. Examples of combinational logic elements include AND, OR, and inverter gates. Sequential elements consist of flip-flops and latches.

When sequential elements are used in scan testing, they are referred to as scan cells. During internal scan testing, a large complex sequential circuit is reduced to many smaller simpler combinational circuits. In a scan chain, each scan cell operates like a direct input or output for one of these simpler combinational circuits. Since the circuits under test are simpler, the task of pattern generation is made easier. In fact, the pattern generation problem is simplified enough that generating test patterns for complex digital designs can be fully automated using commercial off-the-shelf automated test pattern generation (ATPG) tools.

ATPG tools are used to generate scan patterns. ATPG-generated patterns differ from functional test patterns in that they do not necessarily test the functionality of a circuit-under-test. Instead, functionality is verified during the design verification and characterization phases of IC development, and ATPG patterns are created to exercise specific types of device faults that are known to be caused by defects introduced during device manufacturing.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scantheory.2.03

<	>
-----------------	-----------------

SCAN Theory of Operation : Boundary Scan Testing

Boundary Scan Testing

Boundary scan testing refers to a test technique that involves creating a long scan chain around the periphery of a device by chaining together the I/O elements of the design. The predominant boundary scan technique uses a standard chip-board test interface, often referred to as JTAG interface. JTAG is the Joint Test Action Group, the industry group that promoted the standardization of this interface. Today, the industry standard for this test interface is known as the IEEE Standard Test and Boundary Scan Architecture (IEEE Std. 1149.1).

Unlike internal scan techniques, boundary scan techniques do not simplify testing of complex internal device circuits. Instead, boundary scan enables board-level testing through direct access to the input and output pads of the integrated circuits installed on a printed circuit board. However, the JTAG test access port (TAP) supports extended functionality beyond boundary scan testing. Often, device test configuration for internal scan and built-in self test (BIST) modes are controlled using the JTAG TAP.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

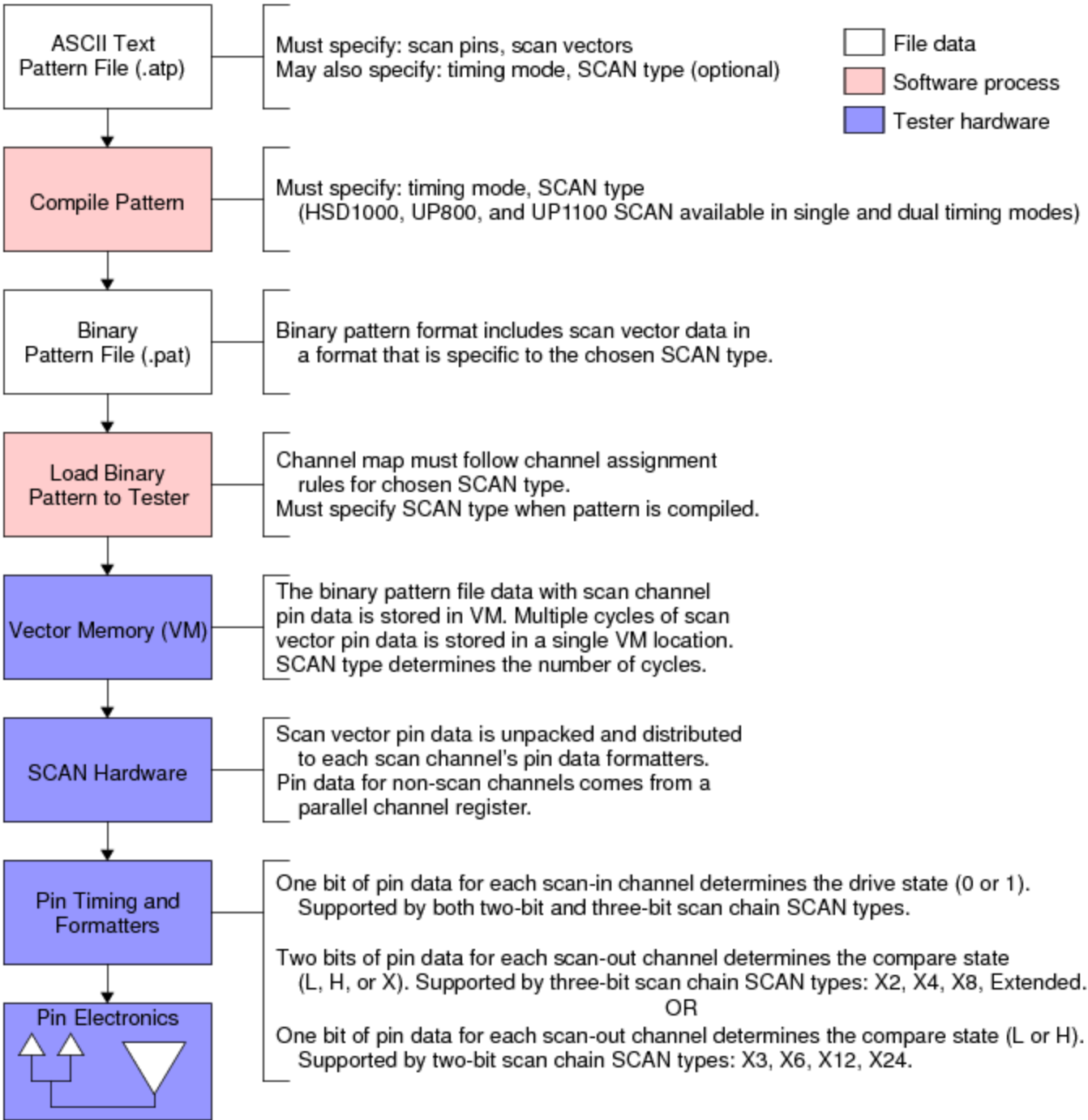
scantheory.2.04

<	>
---	---

[SCAN Theory of Operation](#) : SCAN Functional Flow Chart

SCAN Functional Flow Chart

The following flow chart shows the basic scan testing process using the HSD1000, UltraPin800, or UltraPin1600 digital instrument board.



SCAN Functional Flow Chart

scantheory.2.05



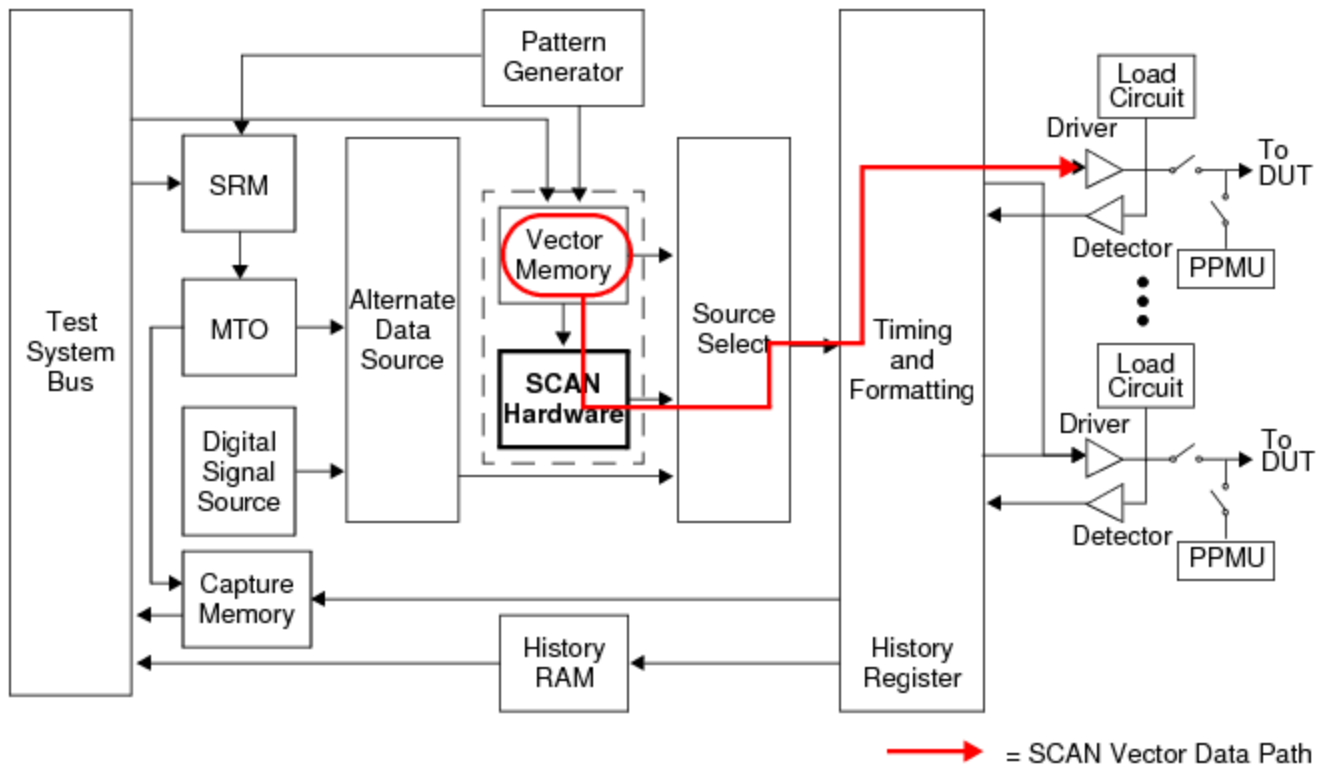
SCAN Theory of Operation : SCAN Hardware and Software

SCAN Hardware and Software

Every HSD1000, UltraPin800, and UltraPin1600 digital channel has the ability to use SCAN hardware and software features to store scan pattern data efficiently and pool vector memory (VM) across several digital channels. These digital instrument boards use a pattern memory sharing technique to store more scan pattern data in VM when the pattern data is made up of scan vectors.

When a non-parallel SCAN type is specified, the pattern compiler effectively compresses scan vector data so that more than one cycle per channel worth of pattern data is stored in a single VM address location.

The SCAN hardware is not a separate scan pattern data memory. After the pattern is loaded and the pattern is running, SCAN hardware is used to unpack the scan vector data from VM. Both the SCAN type, which is specified when the pattern is compiled, and the channel map that is active when the pattern is loaded determines how the data is unpacked and routed to digital channels. When using SCAN hardware for scan testing, pattern data is stored in VM, sent through the SCAN hardware, the source select multiplexer, and the timing and formatting circuitry to the digital channels. The digital channels deliver scan-in pin drive data to the DUT. The scan-out pin expect data is compared against the digital signal from the DUT. This is shown in the block diagram.



Digital Subsystem Block Diagram (two of 64 channels shown)

These digital instrument boards do not use the alternate data source (ADS) bus for routing scan pattern data to the digital channels. Instead, it unpacks SCAN-packed scan vector data using SCAN hardware. The SCAN hardware is not a separate scan pattern data memory. SCAN hardware unpacks scan pattern data stored in VM in a way that takes “decompressing” of scan pattern data. The pattern compiler effectively compresses scan vector data so that more than one cycle worth of pattern data is stored in a single vector memory location per channel.

<	>
---	---

scantheory.2.06

	
---	---

SCAN Theory of Operation : SCAN Regions

SCAN Regions

Each HSD1000 digital channel board has 64 channels that are divided into eight SCAN regions of eight channels each. Each scan region has its own shared VM and SCAN hardware. Since each digital channel board has 64 channels, and a scan chain requires two channels (one for scan in, another for scan out), each channel board can support up to 32 scan chains.

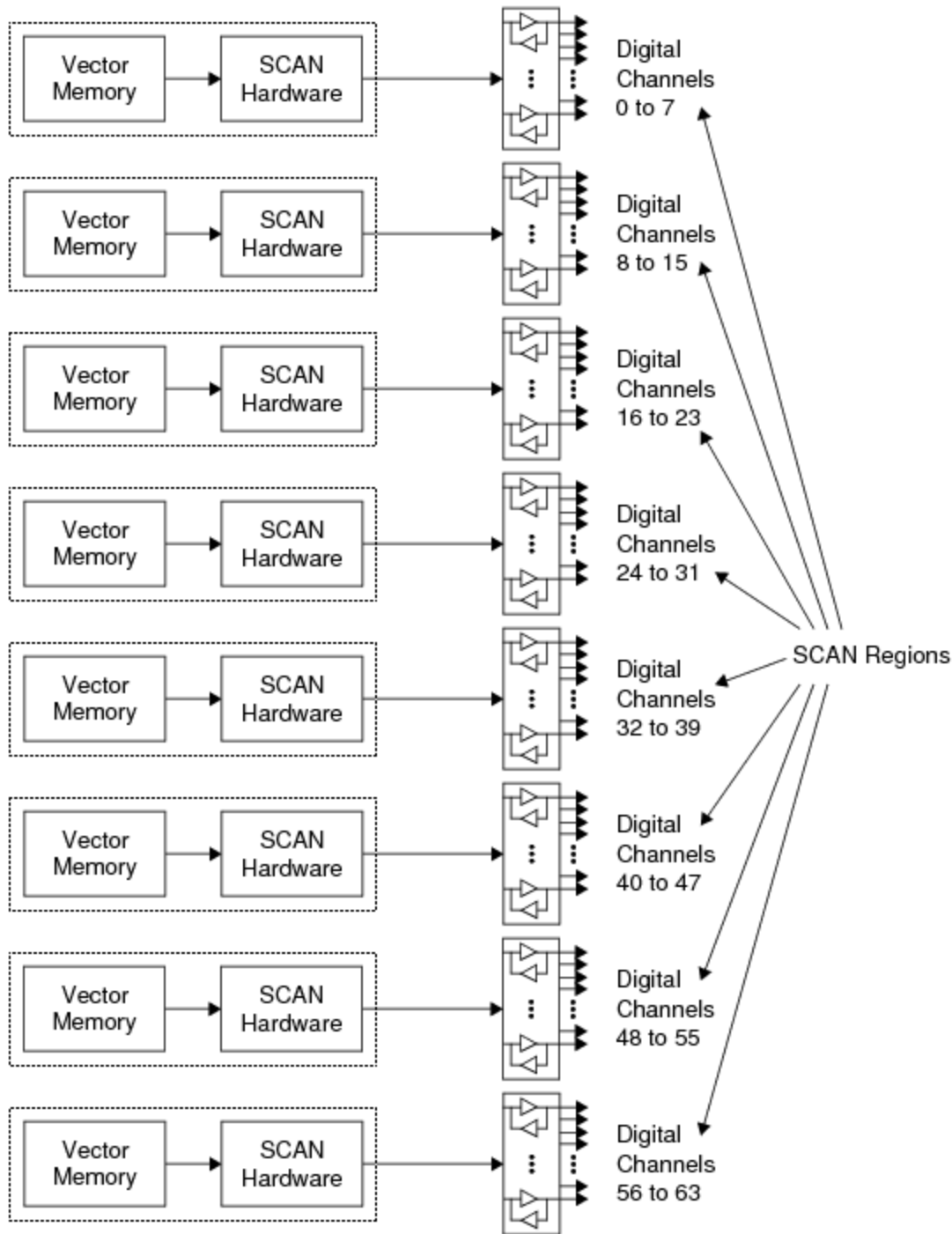
Each UltraPin800 digital channel board has 128 channels that are divided into 16 SCAN regions of eight channels each. Each scan region has its own shared VM and SCAN hardware. Since each digital channel board has 128 channels, and a scan chain requires two channels (one for scan in, another for scan out), each channel board can support up to 64 scan chains.

Each UltraPin1600 digital channel board has 256 channels that are divided into 32 SCAN regions of eight channels each. Each scan region has its own shared VM and SCAN hardware. Since each digital channel board has 256 channels, and a scan chain requires two channels (one for scan in, another for scan out), each channel board can support up to 128 scan chains.

A scan chain can cross boards and regions. For example, a scan-in pin on a chain can be on one board and a scan-out pin for that chain can be on another board.

Note:

This document focuses on scan testing using scan chains that associate a scan-in pin with a scan-out pin. However, HSD1000, and UltraPin800, and UltraPin1600 SCAN also supports serial input applications that use a single scan-in or scan-out pin.



Note that for UltraPin800, this diagram scales up to 128 channels by adding 8 more SCAN regions. For UltraPin1600, you need to add 24 more SCAN regions to scale up to 256 channels.

[HSD1000 SCAN Regions](#)

scantheory.2.07

<	>
-----------------	-----------------

SCAN Theory of Operation : SCAN Vector Memory

SCAN Vector Memory

Scan vector pattern data is stored in the same HSD1000, UltraPin800, or UltraPin1600 VM as regular parallel vector pattern data. The VM that is used for SCAN-packed pattern data is shared across the eight digital channels within the same SCAN region. During a pattern burst, the digital board routes VM pattern data through SCAN hardware where scan vector data is unpacked and routed to the scan channels.

The SCAN type that is defined when the pattern is compiled and the current channel map determine how SCAN pin data is divided and unpacked across channels and pattern cycles. To achieve the maximum scan pattern depth and load SCAN-compiled patterns, you must assign scan channels according to the guidelines for SCAN optimization. For information on maximum number of scan chains and maximum scan depth for three-bit and two-bit scan applications, see [Selecting SCAN Type](#).

<	>
-----------------	-----------------

scantheory.2.08

<	>
-----------------	-----------------

SCAN Theory of Operation : SCAN Types

SCAN Types

When a pattern file uses scan vectors, you must specify a SCAN type to compile the file. The SCAN type determines how the VM is shared and divided across SCAN regions and the scan channels of each region.

This section includes the following topics:

- | | |
|---|----------------------|
| • | Three-Bit SCAN Types |
| • | Two-bit SCAN Types |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

scantheory.2.09

<

>

SCAN Theory of Operation : SCAN Types : Three-Bit SCAN Types

Three-Bit SCAN Types

The SCAN hardware supports the following three-bit SCAN types. The figures for each SCAN type show one parallel test vector and describe how channels are “packed” to achieve the maximum scan chain depth.

☒ **Note:**

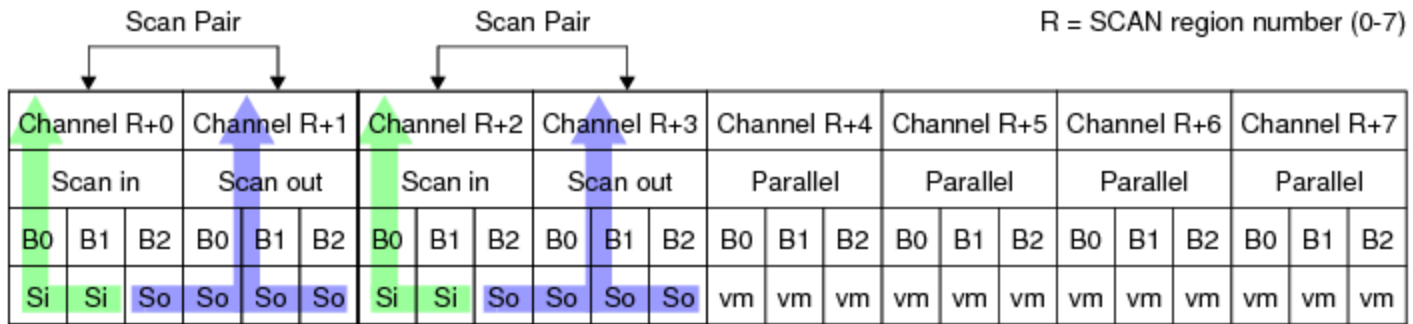
Some of the available memory for scan chains is used for hardware overhead, SRM download images, timing and levels images, and all of the VM patterns, including the one used to burst the scan chains. The exact amount of memory space used for these purposes depends on the specific application. All scan depth values listed here are for dual PatGen mode. For single PatGen mode, these scan depth values would be cut in half.

X2 SCAN Type

X2 SCAN uses two channels’ worth of VM to provide two cycles of scan vector pattern data using the same number of pin data bits that are used for a single cycle of parallel vector pin data.

- Scan-in pins must be on the even channels.
- Scan-out pins must be on the odd channels.
- $128\text{Mcycles} \times 2 = 256\text{Mcycles}$ per scan chain on HSD1000 and UltraPin800
- $256\text{Mcycles} \times 2 = 512\text{Mcycles}$ per scan chain on UltraPin1600
- Up to four scan-in and four scan-out pins can be assigned to a single SCAN region.

R = SCAN region number (0-7)

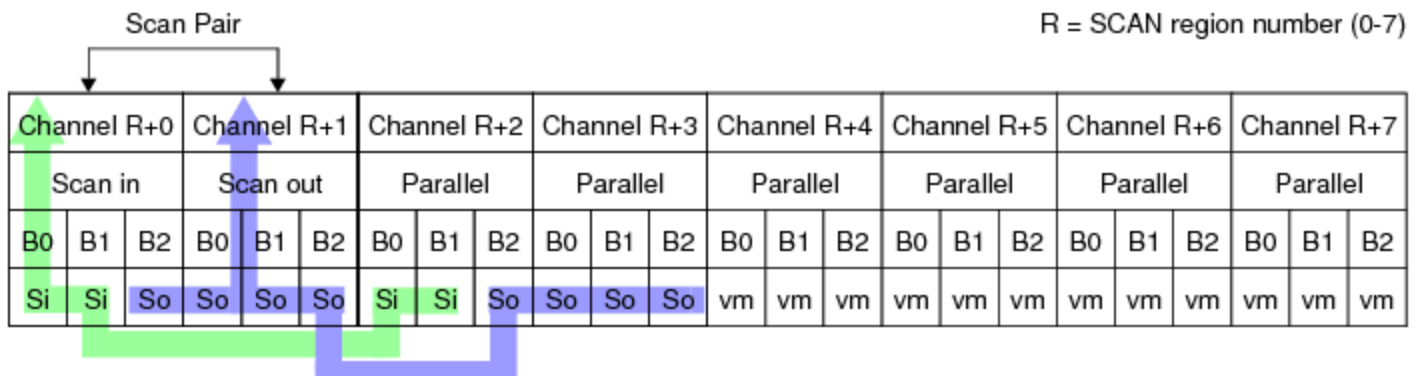


SCAN Region Configured for X2 SCAN

X4 SCAN Type

X4 SCAN uses four channels' worth of VM to provide four cycles of scan vector pattern data using the same number of pin data bits that are used for a single cycle of parallel vector pin data.

- Scan-in pins must be on the even channels.
- Scan-out pins must be on the odd channels.
- $128\text{Mcycles} \times 4 = 512\text{Mcycles}$ per scan chain on HSD1000 and UltraPin800
- $256\text{Mcycles} \times 4 = 1024\text{Mcycles}$ per scan chain on UltraPin1600
- Up to two scan-in and two scan-out pins can be assigned to a single SCAN region.



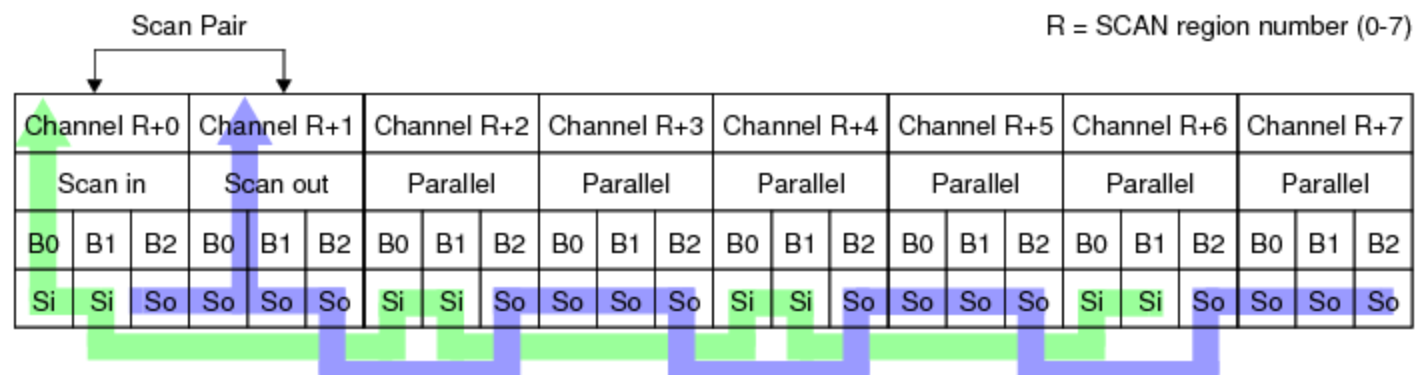
SCAN Region Configured for X4 SCAN

X8 SCAN Type

X8 SCAN uses all eight channels' worth of VM to provide eight cycles of scan vector pattern data using the same amount of VM that is normally used for a single cycle of parallel vector pin data.

- Scan-in pins must be on the even channels.
- Scan-out pins must be on the odd channels.

- 128Mcycles x 8 = 1024Mcycles per scan chain on HSD1000 and UltraPin800
- 256Mcycles x 8 = 2048Mcycles per scan chain on UltraPin1600
- Only one scan-in and one scan-out pin can be assigned to a single SCAN region.



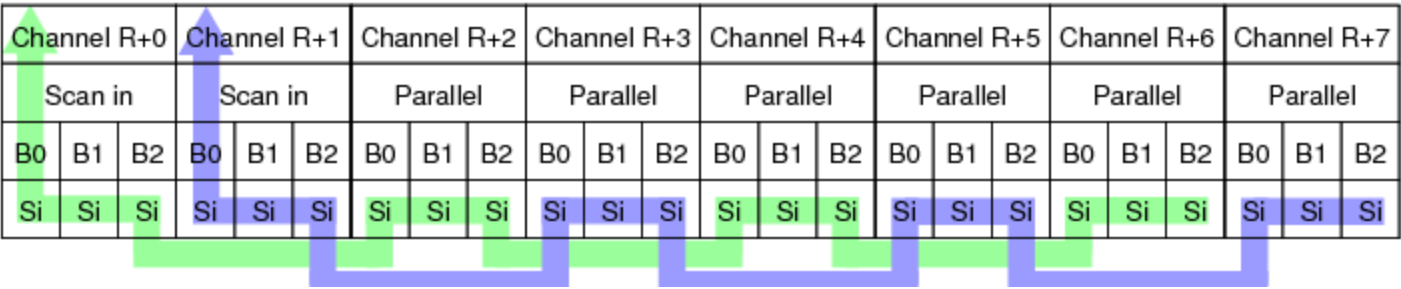
SCAN Region Configured for X8 SCAN

Extended SCAN Type

Extended SCAN uses eight channels' worth of VM to provide 12 cycles of scan vector pattern data using the same number of pin data bits that are used for a single cycle of parallel vector pin data. Unlike other three-bit SCAN types, scan-in and scan-out pins must be divided between SCAN regions.

The following figure shows three-bit scan for the drive channels (scan-in pins), eight channels of VM to two drive channels using the Extended SCAN type. Note the following:

- Two scan-in pins (drive channels) in one SCAN region
- Scan-in on channel 0 and channel 1. Extended SCAN is an exception to the even-odd SCAN channel rule.
- 128Mcycles x 12 = 1536Mcycles per scan chain on HSD1000 and UltraPin800
- 256Mcycles x 12 = 3072Mcycles per scan chain on UltraPin1600
- Up to two scan-in pins per SCAN region

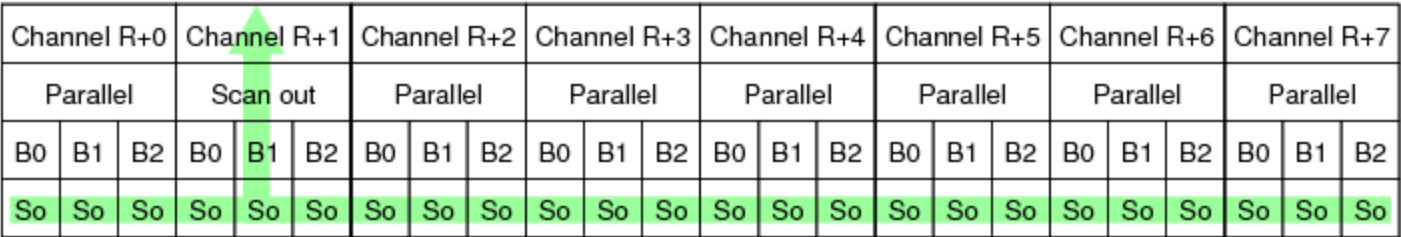


SCAN Region Configured for Extended SCAN Scan-in Pins

The following figure shows three-bit scan for the compare channels (scan-out pins), eight channels of VM to one compare channel using the Extended SCAN type. Note the following:

- One scan-out pin (compare channel) in one SCAN region
- Scan-out on channel R+1
- 128Mcycles x 12 = 1536Mcycles per scan chain on HSD1000 and UltraPin800
- 256Mcycles x 12 = 3072Mcycles per scan chain on UltraPin1600
- Only one scan-out pin per SCAN region

R = region number (0-7)



HSD1000 or UltraPin800 SCAN Region Configured for Extended SCAN Scan-out Pins

<

>

2015 Teradyne, Inc. All rights reserved.

scantheory.2.10

<	>
-----------------	-----------------

SCAN Theory of Operation : SCAN Types : Two-bit SCAN Types

Two-bit SCAN Types

The SCAN hardware supports the following two-bit SCAN types. The figures for each SCAN type show one parallel test vector and describe how channels are “packed” to achieve the maximum scan chain depth.

☒ **Note:**

Some of the available memory for scan chains is used for hardware overhead, SRM download images, timing and levels images, and all of the VM patterns, including the one used to burst the scan chains. The exact amount of memory space used for these purposes depends on the specific application. All scan depth values listed here are for dual PatGen mode. For single PatGen mode, these scan depth values would be cut in half.

X3 SCAN Type

X3 SCAN uses one channel’s worth of VM to provide three cycles of scan vector pattern data using the same number of pin data bits that are used for a single cycle of parallel vector pin data.

- | | |
|--|---|
| | <ul style="list-style-type: none">• Scan-in pins can be assigned to any channel. |
| | <ul style="list-style-type: none">• Scan-out pins can be assigned to any channel. |
| | <ul style="list-style-type: none">• Up to eight scan pins can be assigned to each SCAN region. |
| | <ul style="list-style-type: none">• $128\text{Mcycles} \times 3 = 384\text{Mcycles}$ per scan chain on HSD1000 and UltraPin800 |
| | <ul style="list-style-type: none">• $256\text{Mcycles} \times 3 = 768\text{Mcycles}$ per scan chain on UltraPin1600 |

R = region number (0-7)

Channel R+0			Channel R+1			Channel R+2			Channel R+3			Channel R+4			Channel R+5			Channel R+6			Channel R+7		
Scan in			Scan out			Scan in			Scan out			Parallel			Parallel			Parallel			Parallel		
B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2
Si	Si	Si	So	So	So	Si	Si	Si	So	So	So	vm	vm	vm	vm	vm	vm	vm	vm	vm	vm	vm	vm

SCAN Region Configured for X3 SCAN

X6 SCAN Type

X6 SCAN uses four channels' worth of VM to provide six cycles of scan vector pattern data using the same number of pin data bits that are used for a single cycle of parallel vector pin data.

- Scan-in or scan-out pins can be assigned to the first, second, fifth, or sixth channels of each SCAN region.

- Up to four scan pins can be assigned to each SCAN region.

- $128\text{Mcycles} \times 6 = 768\text{Mcycles}$ per scan chain on HSD1000 and UltraPin800

- $256\text{Mcycles} \times 6 = 1536\text{Mcycles}$ per scan chain on UltraPin1600

R = region number (0-7)

Channel R+0			Channel R+1			Channel R+2			Channel R+3			Channel R+4			Channel R+5			Channel R+6			Channel R+7		
Scan in			Scan out			Parallel			Parallel			Parallel			Parallel			Parallel			Parallel		
B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2
Si	Si	Si	So	So	So	Si	Si	Si	So	So	So	vm	vm	vm	vm	vm	vm	vm	vm	vm	vm	vm	vm

SCAN Region Configured for X6 SCAN

X12 SCAN Type

X12 SCAN uses eight channels' worth of VM to provide 12 cycles of scan vector pattern data using the same number of pin data bits that are used for a single cycle of parallel vector pin data.

- Scan-in or scan-out pins can be assigned to the first or second channels of each SCAN region.

- Up to two scan pins can be assigned to each SCAN region.

- $128\text{Mcycles} \times 12 = 1536\text{Mcycles}$ per scan chain on HSD1000 and UltraPin800

- $256\text{Mcycles} \times 12 = 3072\text{Mcycles}$ per scan chain on UltraPin1600

R = region number (0-7)

Channel R+0			Channel R+1			Channel R+2			Channel R+3			Channel R+4			Channel R+5			Channel R+6			Channel R+7		
Scan in			Scan out			Parallel			Parallel			Parallel			Parallel			Parallel			Parallel		
B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2
Si	Si	Si	So	So	So	Si	Si	Si	So	So	So	Si	Si	Si	So	So	So	Si	Si	Si	So	So	So

SCAN Region Configured for X12 SCAN

X24 SCAN Type

X24 SCAN uses 8 channels' worth of VM to provide 24 cycles of scan vector pattern data using the same number of pin data bits that are used for a single cycle of parallel vector pin data.

The following figure shows two-bit scan for one X24 SCAN channel, eight channels of VM to one compare channel (scan-out pin) or one drive channel (scan-in pin). Note the following:

- Scan-in or scan-out pins must be assigned to the first channels of each SCAN region.

- Only one scan-in pin or one scan-out pin can be assigned to each SCAN region.

- $128\text{Mcycles} \times 24 = 3072\text{Mcycles}$ per scan chain on HSD1000 and UltraPin800

- $256\text{Mcycles} \times 24 = 6144\text{Mcycles}$ per scan chain on UltraPin1600

R = region number (0-7)

Channel R+0			Channel R+1			Channel R+2			Channel R+3			Channel R+4			Channel R+5			Channel R+6			Channel R+7		
Scan data			Parallel			Parallel			Parallel			Parallel			Parallel			Parallel			Parallel		
B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2	B0	B1	B2
So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So	So

SCAN Region Configured for X24 SCAN Scan-in or Scan-out Pins

<		>	
	2015 Teradyne, Inc. All rights reserved.		